The gcodepreview OpenSCAD library*

Author: William F. Adams
willadams at aol dot com

2024/04/12

Abstract

The gcodepreview library allows using OpenSCAD to move a tool in lines and output dxf and G-code files so as to work as a CAD/CAM program for CNC.

Contents

1 readme.md **2**

2 gcodepreview **5**
2.1 Position and Variables . . . 5
2.2 Tools and Changes . . . 10
2.2.1 toolchange . . . 10
2.2.1.1 Normal Tooling . . . 11
2.2.1.2 Tooling for Keyhole Toolpaths . . . 11
2.2.1.3 Thread mills . . . 12
2.2.1.4 Roundover tooling . . . 12
2.2.1.5 Selecting Tools . . . 12
2.2.2 3D Shapes for Tools . . . 13
2.2.2.1 Normal toolshapes . . . 13
2.2.2.2 Concave toolshapes . . . 14
2.2.3 tooldiameter . . . 15
2.3 File Handling . . . 16
2.4 Movement and Cutting . . . 30

3 gcodepreview_template.scad **32**

4 cut2Dshapes and expansion **35**
4.1 Arcs for toolpaths and DXFs . . . 36
4.2 Keyhole toolpath and undercut tooling . . . 39
4.3 Bézier curves in 3 dimensions . . . 42
4.4 Shapes and tool movement . . . 42

5 Other Resources **42**

*This file (`gcodepreview.dtx`) has version number v0.2, last revised 2024/04/12.

1 readme.md

```
⟨rdm⟩ # gcodepreview
⟨rdm⟩
⟨rdm⟩ OpenSCAD library for moving a tool in lines and arcs so as to model
⟨rdm⟩ how a part would be cut using G-Code, so as to allow OpenSCAD to function
⟨rdm⟩ as a compleat CAD/CAM solution for subtractive CNC (mills and routers)
⟨rdm⟩ by writing out G-code (in some cases toolpaths which would not normally
⟨rdm⟩ be feasible), and to write out DXF files which may be imported into a
⟨rdm⟩ traditional CAM program to create toolpaths.
⟨rdm⟩
⟨rdm⟩ ![OpenSCAD Cut Joinery Module](https://raw.githubusercontent.com/WillAdams/gcodep
⟨rdm⟩
⟨rdm⟩ Updated to make use of Python in OpenSCAD:[^rapcad]
⟨rdm⟩
⟨rdm⟩ [^rapcad]: Previous versions had used RapCAD, so as to take advantage of the writ
⟨rdm⟩ command, which has since been re-written in Python.
⟨rdm⟩
⟨rdm⟩ https://pythonscad.org/ (previously this was http://www.guenther-sohler.net/opens
⟨rdm⟩
⟨rdm⟩ A BlockSCAD file for the initial version of the main modules is available at:
⟨rdm⟩
⟨rdm⟩ https://www.blockscad3d.com/community/projects/1244473
⟨rdm⟩
⟨rdm⟩ The project is discussed at:
⟨rdm⟩
⟨rdm⟩ https://forum.makerforums.info/t/g-code-preview-using-openscad-rapcad/85729
⟨rdm⟩
⟨rdm⟩ and
⟨rdm⟩
⟨rdm⟩ https://forum.makerforums.info/t/openscad-and-python-looking-to-finally-
be-resolved/88171
⟨rdm⟩
⟨rdm⟩ and
⟨rdm⟩
⟨rdm⟩ https://willadams.gitbook.io/design-into-3d/programming
⟨rdm⟩
⟨rdm⟩ Since it is now programmed using Literate Programming (.dtx) there is a PDF:
⟨rdm⟩ https://github.com/WillAdams/gcodepreview/blob/main/gcodepreview.pdf
⟨rdm⟩ which includes all of the source code with formatted commentary.
⟨rdm⟩
⟨rdm⟩ The files for this library are:
⟨rdm⟩
⟨rdm⟩  - gcodepreview.py (gcpy) --- the Python core
⟨rdm⟩  - pygcodepreview.scad (pyscad) --- the Python functions wrapped in OpenSCAD
⟨rdm⟩  - gcodepreview.scad(gcpscad) --- OpenSCAD modules and variables
⟨rdm⟩  - gcodepreview_template.scad (gcptmpl) --- example file
⟨rdm⟩  - cut2Dshapes.scad (cut2D) --- code for cutting 2D shapes
⟨rdm⟩
⟨rdm⟩ Place the files in C:\Users\\\~\Documents\OpenSCAD\libraries and call as:[^librar
⟨rdm⟩
⟨rdm⟩ [^libraries]: C:\Users\\\~\Documents\RapCAD\libraries is deprecated since RapCAD
⟨rdm⟩ needed since Python is now used for writing out files)
⟨rdm⟩
```

```
⟨rdm⟩     use <gcodepreview.py>;
⟨rdm⟩     use <pygcodepreview.scad>;
⟨rdm⟩     include <gcodepreview.scad>;
⟨rdm⟩
⟨rdm⟩ Note that it is necessary to use the first two files (this allows loading
⟨rdm⟩ the Python commands and then wrapping them in OpenSCAD commands) and then
⟨rdm⟩ include the last file (which allows using OpenSCAD variables to selectively
⟨rdm⟩ implement the Python commands via their being wrapped in OpenSCAD modules)
⟨rdm⟩ and define variables which match the project and then use commands such as:
⟨rdm⟩
⟨rdm⟩     opengcodefile(Gcode_filename);
⟨rdm⟩     opendxffile(DXF_filename);
⟨rdm⟩
⟨rdm⟩     difference() {
⟨rdm⟩         setupstock(stocklength, stockwidth, stockthickness, zeroheight, stockorigin);
⟨rdm⟩
⟨rdm⟩     movetosafez();
⟨rdm⟩
⟨rdm⟩     toolchange(squaretoolno,speed * square_ratio);
⟨rdm⟩
⟨rdm⟩     begintoolpath(0,0,0.25);
⟨rdm⟩     beginpolyline(0,0,0.25);
⟨rdm⟩
⟨rdm⟩     cutoneaxis_setfeed("Z",-1,plunge*square_ratio);
⟨rdm⟩     addpolyline(stocklength/2,stockwidth/2,-stockthickness);
⟨rdm⟩
⟨rdm⟩     cutwithfeed(stocklength/2,stockwidth/2,-stockthickness,feed);
⟨rdm⟩
⟨rdm⟩     endtoolpath();
⟨rdm⟩     endpolyline();
⟨rdm⟩
⟨rdm⟩     }
⟨rdm⟩
⟨rdm⟩     closegcodefile();
⟨rdm⟩     closedxffile();
⟨rdm⟩
⟨rdm⟩ which makes a G-code file:
⟨rdm⟩
⟨rdm⟩ ![OpenSCAD template G-code file](https://raw.githubusercontent.com/WillAdams/gcodepreview/ma
⟨rdm⟩
⟨rdm⟩ but one which could only be sent to a machine so as to cut only the softest and most
⟨rdm⟩ yielding of materials since it makes a single full-depth pass, and of which has a
⟨rdm⟩ matching DXF which may be imported into a CAM tool --- but which it is not directly
⟨rdm⟩ possible to assign a toolpath in readily available CAM tools (since it varies in depth
⟨rdm⟩ from beginning-to-end).
⟨rdm⟩
⟨rdm⟩ Importing this DXF and actually cutting it is discussed at:
⟨rdm⟩
⟨rdm⟩ https://forum.makerforums.info/t/rewriting-gcodepreview-with-python/88617/14
⟨rdm⟩
⟨rdm⟩ Tool numbers match those of tooling sold by Carbide 3D (ob. discl.,
⟨rdm⟩ I work for them).
⟨rdm⟩
⟨rdm⟩ Comments are included in the G-code to match those expected by CutViewer.
```

```
⟨rdm⟩
⟨rdm⟩ A complete example file is: gcodepreview_template.scad and another example is
⟨rdm⟩ openscad_gcodepreview_cutjoinery.tres.scad which is made from an
⟨rdm⟩ OpenSCAD Graph Editor file:
⟨rdm⟩
⟨rdm⟩ ![OpenSCAD Graph Editor Cut Joinery File](https://raw.githubusercontent.com/WillA
⟨rdm⟩
⟨rdm⟩ Version 0.1 supports setting up stock, origin, rapid positioning, making cuts,
⟨rdm⟩ and writing out matching G-code, and creating a DXF with polylines.
⟨rdm⟩
⟨rdm⟩ Added features since initial upload:
⟨rdm⟩
⟨rdm⟩  - endpolyline(); --- this command allows ending one polyline so as to
⟨rdm⟩                       allow multiple lines in a DXF
⟨rdm⟩  - separate dxf files are written out for each tool where tool is
⟨rdm⟩    ball/square/V and small/large (10/31/23)
⟨rdm⟩  - re-writing as a Literate Program using the LaTeX package docmfp (begun 4/12/24
⟨rdm⟩  - support for additional tooling shapes such as dovetail and keyhole tools
⟨rdm⟩
⟨rdm⟩ Version 0.2 adds support for arcs
⟨rdm⟩
⟨rdm⟩  - DXF: support for arcs (which may be used to make circles) (6/1/24)
⟨rdm⟩  - Specialty toolpaths such as Keyhole which may be used for dovetail as well as
⟨rdm⟩    keyhole cutters
⟨rdm⟩  - Support for curves along the 3rd dimension
⟨rdm⟩  - support for roundover tooling
⟨rdm⟩
⟨rdm⟩ Possible future improvements:
⟨rdm⟩
⟨rdm⟩  - support for additional tooling shapes such as tapered ball-nose tools
⟨rdm⟩    or lollipop cutters or thread-cutting tools
⟨rdm⟩  - G-code: support for G2/G3 arcs and circles
⟨rdm⟩  - G-code: import external tool libraries and feeds and speeds from JSON or CSV f
⟨rdm⟩  - general coding improvements --- current coding style is quite prosaic
⟨rdm⟩  - generalized modules for cutting out various shapes/geometries ---
⟨rdm⟩    an in-process one is to cut a rectangular area as vertical passes
⟨rdm⟩    (the horizontal version will be developed presently)
⟨rdm⟩
⟨rdm⟩ Note for G-code generation that it is up to the user to implement Depth per Pass
⟨rdm⟩ so as to not take a single full-depth pass. Working from a DXF of course allows
⟨rdm⟩ one to off-load such considerations to a specialized CAM tool.
⟨rdm⟩
⟨rdm⟩ Deprecated feature:
⟨rdm⟩
⟨rdm⟩  - exporting SVGs --- while this was begun, it turns out that
⟨rdm⟩    these are written out upside down due to coordinate system differences
⟨rdm⟩    between OpenSCAD/DXFs and SVGs (it is possible that METAPOST will be used
⟨rdm⟩    instead for future versions)
⟨rdm⟩
```

2 gcodepreview

As noted above, this library works by using Python code as a back-end so as to persistently store and access variables, and to write out files. Doing so requires a total of three files:

- A Python file: gcodepreview.py (gcpy) — this will have variables in the traditional sense which may be used for tracking machine position and so forth
- An OpenSCAD file: gcodepreview.scad (gcpscad) — which wraps the Python code in OpenSCAD
- An OpenSCAD file: pygcodepreview.scad (pyscad) — which uses the other two files and which is `included` allowing it to access OpenSCAD variables for branching

Each file will begin with a suitable comment indicating the file type:

```
⟨gcpy⟩#!/usr/bin/env python
⟨gcpy⟩
```

```
⟨pyscad⟩ //!OpenSCAD
⟨pyscad⟩
```

```
⟨gcpscad⟩ //!OpenSCAD
⟨gcpscad⟩
⟨gcpscad⟩ //gcodepreview 0.1
⟨gcpscad⟩ //
⟨gcpscad⟩ //used via use <gcodepreview.py>;
⟨gcpscad⟩ //         use <pygcodepreview.scad>;
⟨gcpscad⟩ //         include <gcodepreview.scad>;
⟨gcpscad⟩ //
```

`writeln` The original implementation in RapSCAD used a command `writeln` — fortunately, this command is easily re-created in Python:

```
⟨gcpy⟩def writeln(*arguments):
⟨gcpy⟩    line_to_write = ""
⟨gcpy⟩    for element in arguments:
⟨gcpy⟩        line_to_write += element
⟨gcpy⟩    f.write(line_to_write)
⟨gcpy⟩    f.write("\n")
```

which command will accept a series of arguments and then write them out to a file object.

2.1 Position and Variables

In modeling the machine motion and G-code it will be necessary to have the machine track several variables. This will be done using paired functions (which will set and return the matching variable) and a matching (global) variable, as well as additional functions for setting the matching variable.

The first such variables are for XYZ position:

mpx • mpx

mpy • mpy

mpz • mpz

Similarly, for some toolpaths it will be necessary to track the depth along the Z-axis as the toolpath is cut out:

tpz • tpz

It will further be necessary to have a variable for the current tool:

currenttool • currenttool

For each intended command it will be necessary to implement an appropriate aspect in each file. The Python file will manage the Python variables and handle things which can only be done in Python, while there will be two OpenSCAD files as noted above, one which calls the Python code (this will be `used`), while the other will be `included` and will be able to access and use OpenSCAD variables, as well as implement Customizer options.

Note that as a convention, where it is necessary for a module to coordinate between Python and OpenSCAD, it will be necessary for there to be three separate divisions: a `p`<foo> Python definition for the manipulation of Python variables and any file routines, an `o`<foo> OpenSCAD module which will wrap up the Python function call, and lastly a <foo> OpenSCAD module which will be `<include>`d so as to be able to make use of OpenSCAD variables.

psetupstock The first such routine will be appropriately enough, to set up the stock, and perform other initializations.

```
173 ⟨gcpy⟩def psetupstock(stocklength, stockwidth, stockthickness, zeroheight, stockorigin)
174 ⟨gcpy⟩    global mpx
175 ⟨gcpy⟩    mpx = float(0)
176 ⟨gcpy⟩    global mpy
177 ⟨gcpy⟩    mpy = float(0)
178 ⟨gcpy⟩    global mpz
179 ⟨gcpy⟩    mpz = float(0)
180 ⟨gcpy⟩    global tpz
181 ⟨gcpy⟩    tpz = float(0)
182 ⟨gcpy⟩    global currenttool
183 ⟨gcpy⟩    currenttool = 102
184 ⟨gcpy⟩
```

osetupstock The intermediary OpenSCAD code simply calls the Python version.

```
185 ⟨pyscad⟩ module osetupstock(stocklength, stockwidth, stockthickness, zeroheight, stockor
186 ⟨pyscad⟩     psetupstock(stocklength, stockwidth, stockthickness, zeroheight, stockorigi
187 ⟨pyscad⟩ }
188 ⟨pyscad⟩
```

setupstock The OpenSCAD code which is called has parameters for the user to to set will create comments in the G-code which set the stock dimensions and its position relative to the origin.

```
⟨gcpscad⟩ module setupstock(stocklength, stockwidth, stockthickness, zeroheight, stockorigin) {
⟨gcpscad⟩   osetupstock(stocklength, stockwidth, stockthickness, zeroheight, stockorigin);
⟨gcpscad⟩ //initialize default tool and XYZ origin
⟨gcpscad⟩   osettool(102);
⟨gcpscad⟩   oset(0,0,0);
⟨gcpscad⟩   if (zeroheight == "Top") {
⟨gcpscad⟩     if (stockorigin == "Lower-Left") {
⟨gcpscad⟩     translate([0, 0, (-stockthickness)]){
⟨gcpscad⟩     cube([stocklength, stockwidth, stockthickness], center=false);
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩ owritethree("(stockMin:0.00mm, 0.00mm, -",str(stockthickness),"mm)");
⟨gcpscad⟩ owritefive("(stockMax:",str(stocklength),"mm, ",str(stockwidth),"mm, 0.00mm)");
⟨gcpscad⟩     owritenine("(STOCK/BLOCK, ",str(stocklength),", ",str(stockwidth),", ",str(stockthic
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩      else if (stockorigin == "Center-Left") {
⟨gcpscad⟩     translate([0, (-stockwidth / 2), -stockthickness]){
⟨gcpscad⟩       cube([stocklength, stockwidth, stockthickness], center=false);
⟨gcpscad⟩     if (generategcode == true) {
⟨gcpscad⟩ owritefive("(stockMin:0.00mm, -",str(stockwidth/2),"mm, -",str(stockthickness),"mm)");
⟨gcpscad⟩ owritefive("(stockMax:",str(stocklength),"mm, ",str(stockwidth/2),"mm, 0.00mm)");
⟨gcpscad⟩     owriteeleven("(STOCK/BLOCK, ",str(stocklength),", ",str(stockwidth),", ",str(stockth
⟨gcpscad⟩     }
⟨gcpscad⟩     }
⟨gcpscad⟩     } else if (stockorigin == "Top-Left") {
⟨gcpscad⟩     translate([0, (-stockwidth), -stockthickness]){
⟨gcpscad⟩       cube([stocklength, stockwidth, stockthickness], center=false);
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩ owritefive("(stockMin:0.00mm, -",str(stockwidth),"mm, -",str(stockthickness),"mm)");
⟨gcpscad⟩ owritethree("(stockMax:",str(stocklength),"mm, 0.00mm, 0.00mm)");
⟨gcpscad⟩ owriteeleven("(STOCK/BLOCK, ",str(stocklength),", ",str(stockwidth),", ",str(stockthickn
⟨gcpscad⟩     }
⟨gcpscad⟩     }
⟨gcpscad⟩     }
⟨gcpscad⟩  else if (stockorigin == "Center") {
⟨gcpscad⟩ //owritecomment("Center");
⟨gcpscad⟩     translate([(-stocklength / 2), (-stockwidth / 2), -stockthickness]){
⟨gcpscad⟩       cube([stocklength, stockwidth, stockthickness], center=false);
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩ owriteseven("(stockMin: -",str(stocklength/2),", -",str(stockwidth/2),"mm, -
  ",str(stockthickness),"mm)");
⟨gcpscad⟩ owritefive("(stockMax:",str(stocklength/2),"mm, ",str(stockwidth/2),"mm, 0.00mm)");
⟨gcpscad⟩ owritethirteen("(STOCK/BLOCK, ",str(stocklength),", ",str(stockwidth),", ",str(stockthic
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩ } else if (zeroheight == "Bottom") {
⟨gcpscad⟩ //owritecomment("Bottom");
⟨gcpscad⟩     if (stockorigin == "Lower-Left") {
⟨gcpscad⟩     cube([stocklength, stockwidth, stockthickness], center=false);
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩ owriteone("(stockMin:0.00mm, 0.00mm, 0.00mm)");
⟨gcpscad⟩ owriteseven("(stockMax:",str(stocklength),"mm, ",str(stockwidth),"mm, ",str(stockthickne
```

```
⟨gcpscad⟩ owriteseven("(STOCK/BLOCK, ",str(stocklength),", ",str(stockwidth),", ",str(st
⟨gcpscad⟩     }
⟨gcpscad⟩ } else if (stockorigin == "Center-Left") {
⟨gcpscad⟩     translate([0, (-stockwidth / 2), 0]){
⟨gcpscad⟩       cube([stocklength, stockwidth, stockthickness], center=false);
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩ owritethree("(stockMin:0.00mm, -",str(stockwidth/2),"mm, 0.00mm)");
⟨gcpscad⟩ owriteseven("(stockMax:",str(stocklength),"mm, ",str(stockwidth/2),"mm, ",str
⟨gcpscad⟩ owritenine("(STOCK/BLOCK, ",str(stocklength),", ",str(stockwidth),", ",str(sto
⟨gcpscad⟩     }
⟨gcpscad⟩     }
⟨gcpscad⟩  } else if (stockorigin == "Top-Left") {
⟨gcpscad⟩     translate([0, (-stockwidth), 0]){
⟨gcpscad⟩       cube([stocklength, stockwidth, stockthickness], center=false);
⟨gcpscad⟩     }
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩ owritethree("(stockMin:0.00mm, -",str(stockwidth),"mm, 0.00mm)");
⟨gcpscad⟩ owritefive("(stockMax:",str(stocklength),"mm, 0.00mm, ",str(stockthickness),"m
⟨gcpscad⟩ owritenine("(STOCK/BLOCK, ",str(stocklength),", ",str(stockwidth),", ",str(sto
⟨gcpscad⟩ }
⟨gcpscad⟩ } else if (stockorigin == "Center") {
⟨gcpscad⟩     translate([(-stocklength / 2), (-stockwidth / 2), 0]){
⟨gcpscad⟩       cube([stocklength, stockwidth, stockthickness], center=false);
⟨gcpscad⟩     }
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩ owritefive("(stockMin:-",str(stocklength/2),", -",str(stockwidth/2),"mm, 0.00m
⟨gcpscad⟩ owriteseven("(stockMax:",str(stocklength/2),"mm, ",str(stockwidth/2),"mm, ",st
⟨gcpscad⟩ owriteeleven("(STOCK/BLOCK, ",str(stocklength),", ",str(stockwidth),", ",str(s
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩  owriteone("G90");
⟨gcpscad⟩  owriteone("G21");
⟨gcpscad⟩ // owriteone("(Move to safe Z to avoid workholding)");
⟨gcpscad⟩ // owriteone("G53G0Z-5.000");
⟨gcpscad⟩ }
⟨gcpscad⟩ //owritecomment("ENDSETUP");
⟨gcpscad⟩ }
⟨gcpscad⟩
```

xpos
ypos
zpos

It will be necessary to have Python functions which return the current values of the machine position in Cartesian coordinates:

```
⟨gcpy⟩def xpos():
⟨gcpy⟩    global mpx
⟨gcpy⟩    return mpx
⟨gcpy⟩
⟨gcpy⟩def ypos():
⟨gcpy⟩    global mpy
⟨gcpy⟩    return mpy
⟨gcpy⟩
⟨gcpy⟩def zpos():
⟨gcpy⟩    global mpz
⟨gcpy⟩    return mpz
```

```
⟨gcpy⟩
⟨gcpy⟩def tzpos():
⟨gcpy⟩    global tpz
⟨gcpy⟩    return tpz
⟨gcpy⟩
```

psetxpos
psetypos
psetzpos
psettzpos

and in turn, functions which set the positions:

```
⟨gcpy⟩def psetxpos(newxpos):
⟨gcpy⟩    global mpx
⟨gcpy⟩    mpx = newxpos
⟨gcpy⟩
⟨gcpy⟩def psetypos(newypos):
⟨gcpy⟩    global mpy
⟨gcpy⟩    mpy = newypos
⟨gcpy⟩
⟨gcpy⟩def psetzpos(newzpos):
⟨gcpy⟩    global mpz
⟨gcpy⟩    mpz = newzpos
⟨gcpy⟩
⟨gcpy⟩def psettzpos(newtzpos):
⟨gcpy⟩    global tpz
⟨gcpy⟩    tpz = newtzpos
⟨gcpy⟩
```

getxpos
getypos
getzpos
gettzpos

and as noted above, there will need to be matching OpenSCAD versions.

Note that for routines where the variable is directly passed from OpenSCAD to Python it is possible to have OpenSCAD directly call the matching Python module with no need to use an intermediary OpenSCAD command.

setxpos
setypos
setzpos
settzpos

```
⟨pyscad⟩ function getxpos() = xpos();
⟨pyscad⟩ function getypos() = ypos();
⟨pyscad⟩ function getzpos() = zpos();
⟨pyscad⟩ function gettzpos() = tzpos();
⟨pyscad⟩
⟨pyscad⟩ module setxpos(newxpos) {
⟨pyscad⟩ psetxpos(newxpos);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module setypos(newypos) {
⟨pyscad⟩ psetypos(newypos);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module setzpos(newzpos) {
⟨pyscad⟩ psetzpos(newzpos);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module settzpos(newtzpos) {
⟨pyscad⟩ psettzpos(newtzpos);
⟨pyscad⟩ }
⟨pyscad⟩
```

oset

```
⟨gcpscad⟩ module oset(ex, ey, ez) {
⟨gcpscad⟩ setxpos(ex);
⟨gcpscad⟩ setypos(ey);
```

```
⟨gcpscad⟩ setzpos(ez);
⟨gcpscad⟩ }
⟨gcpscad⟩
```

osettz

```
⟨gcpscad⟩ module osettz(tz) {
⟨gcpscad⟩ settzpos(tz);
⟨gcpscad⟩ }
⟨gcpscad⟩
```

2.2 Tools and Changes

pcurrenttool psettool Similarly Python functions and variables will be used to track and set and return the current tool:

```
⟨gcpy⟩def psettool(tn):
⟨gcpy⟩    global currenttool
⟨gcpy⟩    currenttool = tn
⟨gcpy⟩
⟨gcpy⟩def pcurrent_tool():
⟨gcpy⟩    global currenttool
⟨gcpy⟩    return currenttool
⟨gcpy⟩
```

osettool currenttool and matching OpenSCAD modules set and return the current tool:

```
⟨pyscad⟩ module osettool(tn){
⟨pyscad⟩ psettool(tn);}
⟨pyscad⟩
⟨pyscad⟩ function current_tool() = pcurrent_tool();
⟨pyscad⟩
```

2.2.1 toolchange

toolchange and apply the appropriate commands for a `toolchange`. Note that it is expected that this subsubsection will be updated as needed when new tooling is introduced as additional modules which require specific tooling are added below.

Note that the comments written out in G-code correspond to that used by the G-code previewing tool CutViewer (which is unfortunately, no longer readiily available).

It is possible that rather than hard-coding the tool definitions, a future update will instead read them in from an external file — the .csv format used for tool libraries in Carbide Create seems a likely candidate and worth exploring.

Note that there are many varieties of tooling and not all will be implemented

in the initial version of this project

2.2.1.1 Normal Tooling Most tooling has quite standard shapes and are defined by their profile:

- Square (#201 and 102) — able to cut a flat bottom, perpendicular side and right angle their simple and easily understood geometry makes them a standard choice (a radiused form with a flat bottom, often described as a "bowl bit" is not implemented in this version)

- Ballnose (#202 and 101) — rounded, they are the standard choice for rounded and organic shapes

- V tooling (#301, 302 and 390) — pointed at the tip, they are available in a variety angles and diameters and may be used for decorative V carving, or for chamfering or cutting specific angles (note that the commonly available radiused form is not implemented at this time, see #501 and 502)

```
⟨gcpscad⟩ module toolchange(tool_number,speed) {
⟨gcpscad⟩    osettool(tool_number);
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩  writecomment("Toolpath");
⟨gcpscad⟩  owriteone("M05");
⟨gcpscad⟩ // writecomment("Move to safe Z to avoid workholding");
⟨gcpscad⟩ // owriteone("G53G0Z-5.000");
⟨gcpscad⟩ //    writecomment("Begin toolpath");
⟨gcpscad⟩   if (tool_number == 201) {
⟨gcpscad⟩  writecomment("TOOL/MILL,6.35, 0.00, 0.00, 0.00");
⟨gcpscad⟩   } else if (tool_number == 202) {
⟨gcpscad⟩  writecomment("TOOL/MILL,6.35, 3.17, 0.00, 0.00");
⟨gcpscad⟩   } else if (tool_number == 102) {
⟨gcpscad⟩  writecomment("TOOL/MILL,3.17, 0.00, 0.00, 0.00");
⟨gcpscad⟩   } else if (tool_number == 101) {
⟨gcpscad⟩  writecomment("TOOL/MILL,3.17, 1.58, 0.00, 0.00");
⟨gcpscad⟩   } else if (tool_number == 301) {
⟨gcpscad⟩  writecomment("TOOL/MILL,0.03, 0.00, 6.35, 45.00");
⟨gcpscad⟩   } else if (tool_number == 302) {
⟨gcpscad⟩  writecommment("TOOL/MILL,0.03, 0.00, 10.998, 30.00");
⟨gcpscad⟩   } else if (tool_number == 390) {
⟨gcpscad⟩  writecomment("TOOL/MILL,0.03, 0.00, 1.5875, 45.00");
```

2.2.1.2 Tooling for Keyhole Toolpaths Keyhole toolpaths (see: subsection 4.2 are intended for use with tooling which projects beyond the the narrower shaft and so will cut usefully underneath the visible surface. Also described as "undercut" tooling, but see below.

There are several notable candidates for such tooling:

- Keyhole tools — intended to cut slots for retaining hardware used for picture hanging, they may be used to create slots for other purposes
- Dovetail cutters — used for the joinery of the same name, they cut a large area at the bottom which slants up to a narrower region at a defined angle
- Lollipop cutters — normally used for 3D work, as their name suggests they are essentially a (cutting) ball on a narrow stick (the tool shaft), they are mentioned here only for compleatness' sake and are not (at this time) implemented

```
⟨gcpscad⟩   } else if (tool_number == 375) {
⟨gcpscad⟩  writecomment("TOOL/MILL,9.53, 0.00, 3.17, 0.00");
⟨gcpscad⟩   } else if (tool_number == 814) {
⟨gcpscad⟩  writecomment("TOOL/MILL,12.7, 6.367, 12.7, 0.00");
```

2.2.1.3 Thread mills The implementation of arcs cutting along the Z-axis raises the possibility of cutting threads using "thread mills". See: https://community.carbide3d.com/t/thread-milling-in-metal-on-the-shapeoko-3/5332

Note that it will be necessary to to define modules (see below) for each tool shape.

With the tools delineated, the module is closed out and the tooling information written into the G-code.

```
⟨gcpscad⟩   }
⟨gcpscad⟩     select_tool(tool_number);
⟨gcpscad⟩  owritetwo("M6T",str(tool_number));
⟨gcpscad⟩  owritetwo("M03S",str(speed));
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩
```

2.2.1.4 Roundover tooling It is not possible to represent all tools using tool changes as coded above which require using a `hull` operation between 3D representations of the tools at the beginning and end points.

2.2.1.5 Selecting Tools There must also be a module for selecting tools: `select_tool` which will select the matching module for 3D modeling and pass the appropriate parameters to that module:

`selecttool tool number`

```
⟨gcpscad⟩ module select_tool(tool_number) {
⟨gcpscad⟩ //echo(tool_number);
⟨gcpscad⟩   if (tool_number == 201) {
⟨gcpscad⟩     gcp_endmill_square(6.35, 19.05);
⟨gcpscad⟩   } else if (tool_number == 202) {
⟨gcpscad⟩     gcp_endmill_ball(6.35, 19.05);
⟨gcpscad⟩   } else if (tool_number == 102) {
⟨gcpscad⟩     gcp_endmill_square(3.175, 19.05);
⟨gcpscad⟩   } else if (tool_number == 101) {
⟨gcpscad⟩     gcp_endmill_ball(3.175, 19.05);
```

```
⟨gcpscad⟩   } else if (tool_number == 301) {
⟨gcpscad⟩     gcp_endmill_v(90, 12.7);
⟨gcpscad⟩   } else if (tool_number == 302) {
⟨gcpscad⟩     gcp_endmill_v(60, 12.7);
⟨gcpscad⟩   } else if (tool_number == 390) {
⟨gcpscad⟩     gcp_endmill_v(90, 3.175);
```

For a keyhole tool:

```
⟨gcpscad⟩   } else if (tool_number == 375) {
⟨gcpscad⟩     gcp_keyhole(9.525, 3.175);
```

and dovetail tool:

```
⟨gcpscad⟩   } else if (tool_number == 814) {
⟨gcpscad⟩     gcp_dovetail(12.7, 6.367, 12.7, 14);
```

```
⟨gcpscad⟩   }
⟨gcpscad⟩ }
⟨gcpscad⟩
```

2.2.2 3D Shapes for Tools

Each tool must be modeled in 3D using an OpenSCAD module.

2.2.2.1 Normal toolshapes Most tools are easily implemented with concise 3D descriptions which may be connected with a simple `hull` operation:

gcp endmill square

```
⟨gcpscad⟩ module gcp_endmill_square(es_diameter, es_flute_length) {
⟨gcpscad⟩   cylinder(r1=(es_diameter / 2), r2=(es_diameter / 2), h=es_flute_length, center=false);
⟨gcpscad⟩ }
⟨gcpscad⟩
```

gcp keyhole

```
⟨gcpscad⟩ module gcp_keyhole(es_diameter, es_flute_length) {
⟨gcpscad⟩   cylinder(r1=(es_diameter / 2), r2=(es_diameter / 2), h=es_flute_length, center=false);
⟨gcpscad⟩ }
⟨gcpscad⟩
```

gcp dovetail

```
⟨gcpscad⟩ module gcp_dovetail(dt_bottomdiameter, dt_topdiameter, dt_height, dt_angle) {
⟨gcpscad⟩   cylinder(r1=(dt_bottomdiameter / 2), r2=(dt_topdiameter / 2), h= dt_height, center=fal
⟨gcpscad⟩ }
⟨gcpscad⟩
```

gcp endmill ball

```
⟨gcpscad⟩ module gcp_endmill_ball(es_diameter, es_flute_length) {
⟨gcpscad⟩   translate([0, 0, (es_diameter / 2)]){
⟨gcpscad⟩     union(){
⟨gcpscad⟩       sphere(r=(es_diameter / 2));
⟨gcpscad⟩       cylinder(r1=(es_diameter / 2), r2=(es_diameter / 2), h=es_flute_length, center=fal
⟨gcpscad⟩     }
⟨gcpscad⟩   }
⟨gcpscad⟩ }
⟨gcpscad⟩
```

gcp endmill v

```
⟨gcpscad⟩ module gcp_endmill_v(es_v_angle, es_diameter) {
⟨gcpscad⟩   union(){
⟨gcpscad⟩     cylinder(r1=0, r2=(es_diameter / 2), h=((es_diameter / 2) / tan((es_v_angl
⟨gcpscad⟩     translate([0, 0, ((es_diameter / 2) / tan((es_v_angle / 2)))]){
⟨gcpscad⟩       cylinder(r1=(es_diameter / 2), r2=(es_diameter / 2), h=((es_diameter * 8
⟨gcpscad⟩     }
⟨gcpscad⟩   }
⟨gcpscad⟩ }
⟨gcpscad⟩
```

2.2.2.2 Concave toolshapes While normal tooling may be represented with a single `hull` operation betwixt two 3D toolshapes, concave tooling such as roundover/radius tooling require multiple slices of the tool shape which are then `hull`ed together. Something of this can be seen in the manual work-around for previewing them: https://community.carbide3d.com/t/using-unsupported-tooling-in-carbid 43723.

Ideally, it would be possible to simply identify such tooling using the tool # in the code used for normal toolshapes as above, but the most expedient option is to simply use a specific command for this. Since such tooling is quite limited in its use and normally only used at the surface of the part along an edge, this separation is easily justified.

Because it is necessary to divide the tooling into vertical slices and call the hull operation for each slice the tool definitions are tightly coupled with the module. Note that there are two different modules, the public-facing version which includes the tool number:

radiuscut

```
⟨gcpscad⟩ module radiuscut(bx, by, bz, ex, ey, ez, radiustn) {
⟨gcpscad⟩     if (radiustn == 56125) {
⟨gcpscad⟩         radiuscuttool(bx, by, bz, ex, ey, ez, 0.508/2, 1.531);
⟨gcpscad⟩     } else if (radiustn == 56142) {
⟨gcpscad⟩         radiuscuttool(bx, by, bz, ex, ey, ez, 0.508/2, 2.921);
⟨gcpscad⟩     } else if (radiustn == 312) {
⟨gcpscad⟩         radiuscuttool(bx, by, bz, ex, ey, ez, 1.524/2, 3.175);
⟨gcpscad⟩     } else if (radiustn == 1570) {
⟨gcpscad⟩         radiuscuttool(bx, by, bz, ex, ey, ez, 0.507/2, 4.509);
⟨gcpscad⟩     }
⟨gcpscad⟩ }
⟨gcpscad⟩
```

which then calls the actual `radiuscuttool` module passing in the tip radius and the radius of the rounding. Note that this module sets its quality relative to the value of `$fn`.

```
⟨gcpscad⟩ module radiuscuttool(bx, by, bz, ex, ey, ez, tool_radius_tip, tool_radius_widt
⟨gcpscad⟩ n = 90 + $fn*3;
⟨gcpscad⟩ step = 360/n;
⟨gcpscad⟩
⟨gcpscad⟩ hull(){
⟨gcpscad⟩     translate([bx,by,bz])
⟨gcpscad⟩     cylinder(step,tool_radius_tip,tool_radius_tip);
⟨gcpscad⟩     translate([ex,ey,ez])
⟨gcpscad⟩     cylinder(step,tool_radius_tip,tool_radius_tip);
⟨gcpscad⟩     }
```

```
⟨gcpscad⟩
⟨gcpscad⟩ hull(){
⟨gcpscad⟩ translate([bx,by,bz+tool_radius_width])
⟨gcpscad⟩ cylinder(tool_radius_width*2,tool_radius_tip+tool_radius_width,tool_radius_tip+tool_radi
⟨gcpscad⟩
⟨gcpscad⟩ translate([ex,ey,ez+tool_radius_width])
⟨gcpscad⟩   cylinder(tool_radius_width*2,tool_radius_tip+tool_radius_width,tool_radius_tip+tool_ra
⟨gcpscad⟩     }
⟨gcpscad⟩
⟨gcpscad⟩ for (i=[0:step:90]) {
⟨gcpscad⟩     angle = i;
⟨gcpscad⟩     dx = tool_radius_width*cos(angle);
⟨gcpscad⟩     dxx = tool_radius_width*cos(angle+step);
⟨gcpscad⟩     dzz = tool_radius_width*sin(angle);
⟨gcpscad⟩     dz = tool_radius_width*sin(angle+step);
⟨gcpscad⟩     dh = dz-dzz;
⟨gcpscad⟩     hull(){
⟨gcpscad⟩     translate([bx,by,bz+dz])
⟨gcpscad⟩        cylinder(dh,tool_radius_tip+tool_radius_width-dx,tool_radius_tip+tool_radius_widt
    dxx);
⟨gcpscad⟩     translate([ex,ey,ez+dz])
⟨gcpscad⟩        cylinder(dh,tool_radius_tip+tool_radius_width-dx,tool_radius_tip+tool_radius_widt
    dxx);
⟨gcpscad⟩         }
⟨gcpscad⟩     }
⟨gcpscad⟩ }
⟨gcpscad⟩
```

2.2.3 tooldiameter

It will also be necessary to be able to provide the diameter of the current tool. Arguably, this would be much easier using an object-oriented programming style/dot notation.

One aspect of tool parameters which will need to be supported is shapes which create different profiles based on how deeply the tool is cutting into the surface of the material at a given point. To accommodate this, it will be necessary to either track the thickness of uncut material at any given point, or, to specify the depth of cut as a parameter which is what the initial version will implement.

`tool diameter` The public-facing OpenSCAD code simply calls the matching OpenSCAD module which wraps the Python code:

```
⟨gcpscad⟩  function tool_diameter(td_tool, td_depth) = otool_diameter(td_tool, td_depth);
```

`otool diameter` the matching OpenSCAD function calls the Python function:

```
⟨pyscad⟩  function otool_diameter(td_tool, td_depth) = ptool_diameter(td_tool, td_depth);
```

`ptool diameter` the Python code returns appropriate values based on the specified tool number and depth:

```
⟨gcpy⟩def ptool_diameter(ptd_tool, ptd_depth):
⟨gcpy⟩    if ptd_tool == 201:
⟨gcpy⟩        return 6.35
⟨gcpy⟩    if ptd_tool == 202:
⟨gcpy⟩        if ptd_depth > 3.175:
⟨gcpy⟩            return 6.35
```

```
⟨gcpy⟩        else:
⟨gcpy⟩            return 0
⟨gcpy⟩    if ptd_tool == 102:
⟨gcpy⟩        return 3.175
⟨gcpy⟩    if ptd_tool == 101:
⟨gcpy⟩        if ptd_depth > 1.5875:
⟨gcpy⟩            return 3.175
⟨gcpy⟩        else:
⟨gcpy⟩            return 0
⟨gcpy⟩    if ptd_tool == 301:
⟨gcpy⟩        return 0
⟨gcpy⟩    if ptd_tool == 302:
⟨gcpy⟩        return 0
⟨gcpy⟩    if ptd_tool == 390:
⟨gcpy⟩        return 0
⟨gcpy⟩    if ptd_tool == 375:
⟨gcpy⟩        if ptd_depth < 6.35:
⟨gcpy⟩            return 9.525
⟨gcpy⟩        else:
⟨gcpy⟩            return 6.35
⟨gcpy⟩    if ptd_tool == 814:
⟨gcpy⟩        if ptd_depth > 12.7:
⟨gcpy⟩            return 6.35
⟨gcpy⟩        else:
⟨gcpy⟩            return 12.7
⟨gcpy⟩
```

(Note that 0 values will need to bre replaced with appropriate code.)

2.3 File Handling

For writing to files it will be necessary to have commands for each step of working with the files.

popengcodefile
popendxffile
popendxlgblffile
popendxflgsqfile
popendxflgVfile
popendxfsmblfile
popendxfsmsqfile
popendxfsmVfile
popensvgfile

There is a separate function for each type of file, and for DXFs, there are multiple file instances, one for each combination of different type and size of tool which it is expected a project will work with. Each such file will be suffixed with the tool number.

```
⟨gcpy⟩def popengcodefile(fn):
⟨gcpy⟩    global f
⟨gcpy⟩    f = open(fn, "w")
⟨gcpy⟩
⟨gcpy⟩def popendxffile(fn):
⟨gcpy⟩    global dxf
⟨gcpy⟩    dxf = open(fn, "w")
⟨gcpy⟩
⟨gcpy⟩def popendxlgblffile(fn):
⟨gcpy⟩    global dxflgbl
⟨gcpy⟩    dxflgbl = open(fn, "w")
⟨gcpy⟩
⟨gcpy⟩def popendxflgsqfile(fn):
⟨gcpy⟩    global dxfldsq
⟨gcpy⟩    dxflgsq = open(fn, "w")
⟨gcpy⟩
⟨gcpy⟩def popendxflgVfile(fn):
```

```
⟨gcpy⟩    global dxflgV
⟨gcpy⟩    dxflgV = open(fn, "w")
⟨gcpy⟩
⟨gcpy⟩def popendxfsmblfile(fn):
⟨gcpy⟩    global dxfsmbl
⟨gcpy⟩    dxfsmbl = open(fn, "w")
⟨gcpy⟩
⟨gcpy⟩def popendxfsmsqfile(fn):
⟨gcpy⟩    global dxfsmsq
⟨gcpy⟩    dxfsmsq = open(fn, "w")
⟨gcpy⟩
⟨gcpy⟩def popendxfsmVfile(fn):
⟨gcpy⟩    global dxfsmV
⟨gcpy⟩    dxfsmV = open(fn, "w")
⟨gcpy⟩
⟨gcpy⟩def popendxfKHfile(fn):
⟨gcpy⟩    global dxfKH
⟨gcpy⟩    dxfKH = open(fn, "w")
⟨gcpy⟩
⟨gcpy⟩def popendxDTfile(fn):
⟨gcpy⟩    global dxfDT
⟨gcpy⟩    dxfDT = open(fn, "w")
⟨gcpy⟩
⟨gcpy⟩def popensvgfile(fn):
⟨gcpy⟩    global svg
⟨gcpy⟩    svg = open(fn, "w")
⟨gcpy⟩
```

oopengcodefile
oopensvgfile
oopendxffile

There will need to be matching OpenSCAD modules for the Python functions.

```
⟨pyscad⟩ module oopengcodefile(fn) {
⟨pyscad⟩  popengcodefile(fn);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oopensvgfile(fn) {
⟨pyscad⟩  popensvgfile(fn);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oopendxffile(fn) {
⟨pyscad⟩     echo(fn);
⟨pyscad⟩  popendxffile(fn);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oopendxflgblfile(fn) {
⟨pyscad⟩  popendxflgblfile(fn);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oopendxflgsqfile(fn) {
⟨pyscad⟩  popendxflgsqfile(fn);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oopendxflgVfile(fn) {
⟨pyscad⟩  popendxflgVfile(fn);
⟨pyscad⟩ }
⟨pyscad⟩
```

```
⟨pyscad⟩ module oopendxfsmblfile(fn) {
⟨pyscad⟩  popendxfsmblfile(fn);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oopendxfsmsqfile(fn) {
⟨pyscad⟩     echo(fn);
⟨pyscad⟩  popendxfsmsqfile(fn);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oopendxfsmVfile(fn) {
⟨pyscad⟩  popendxfsmVfile(fn);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oopendxfKHfile(fn) {
⟨pyscad⟩  popendxfKHfile(fn);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oopendxfDTfile(fn) {
⟨pyscad⟩  popendxfDTfile(fn);
⟨pyscad⟩ }
⟨pyscad⟩
```

`opengcodefile` `opensvgfile`

Which have Matching OpenSCAD commands:

```
⟨gcpscad⟩ module opengcodefile(fn) {
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩  oopengcodefile(fn);
⟨gcpscad⟩     echo(fn);
⟨gcpscad⟩     owritecomment(fn);
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module opensvgfile(fn) {
⟨gcpscad⟩ if (generatesvg == true) {
⟨gcpscad⟩  oopensvgfile(fn);
⟨gcpscad⟩     echo(fn);
⟨gcpscad⟩     svgwriteone(str("<?xml version=",chr(34),"1.0",chr(34)," encoding=",chr(34
8",chr(34)," standalone=",chr(34),"no",chr(34),"?> "));
⟨gcpscad⟩ // writesvglineend();
⟨gcpscad⟩ svgwriteone(str("<svg  version=",chr(34),"1.1",chr(34)," xmlns=",chr(34),"http
⟨gcpscad⟩ //<path d="M755.906 0 L755.906 377.953 L0 377.953 L0 0 L755.906 0 Z " stroke="
width="1" fill="none" />
⟨gcpscad⟩ svgwriteone(str("<path d=",chr(34),"M",stocklength*3.77953," 0 L",stocklength*
width=",chr(34),"1",chr(34)," fill=",chr(34),"none",chr(34)," /> "));
⟨gcpscad⟩     }
⟨gcpscad⟩ }
⟨gcpscad⟩
```

`opendxffile`

For each DXF file, in addition to opening the file in the file system there will need to be a Preamble

```
⟨gcpscad⟩ module opendxffile(fn) {
⟨gcpscad⟩ if (generatedxf == true) {
⟨gcpscad⟩  oopendxffile(str(fn,".dxf"));
⟨gcpscad⟩ //    echo(fn);
⟨gcpscad⟩     dxfwriteone("0");
```

```
⟨gcpscad⟩     dxfwriteone("SECTION");
⟨gcpscad⟩     dxfwriteone("2");
⟨gcpscad⟩     dxfwriteone("ENTITIES");
⟨gcpscad⟩ if (large_ball_tool_no >  0) { oopendxflgblfile(str(fn,".",large_ball_tool_no,".dxf"));
⟨gcpscad⟩     dxfpreamble(large_ball_tool_no);
⟨gcpscad⟩ }
⟨gcpscad⟩ if (large_square_tool_no >  0) { oopendxflgsqfile(str(fn,".",large_square_tool_no,".dxf"
⟨gcpscad⟩     dxfpreamble(large_square_tool_no);
⟨gcpscad⟩ }
⟨gcpscad⟩ if (large_V_tool_no >  0) { oopendxflgVfile(str(fn,".",large_V_tool_no,".dxf"));
⟨gcpscad⟩     dxfpreamble(large_V_tool_no);
⟨gcpscad⟩ }
⟨gcpscad⟩ if (small_ball_tool_no >  0) { oopendxfsmblfile(str(fn,".",small_ball_tool_no,".dxf"));
⟨gcpscad⟩     dxfpreamble(small_ball_tool_no);
⟨gcpscad⟩ }
⟨gcpscad⟩ if (small_square_tool_no >  0) { oopendxfsmsqfile(str(fn,".",small_square_tool_no,".dxf"
⟨gcpscad⟩ //    echo(str("tool no",small_square_tool_no));
⟨gcpscad⟩     dxfpreamble(small_square_tool_no);
⟨gcpscad⟩ }
⟨gcpscad⟩ if (small_V_tool_no >  0) { oopendxfsmVfile(str(fn,".",small_V_tool_no,".dxf"));
⟨gcpscad⟩     dxfpreamble(small_V_tool_no);
⟨gcpscad⟩ }
⟨gcpscad⟩ if (KH_tool_no >  0) { oopendxfKHfile(str(fn,".",KH_tool_no,".dxf"));
⟨gcpscad⟩     dxfpreamble(KH_tool_no);
⟨gcpscad⟩ }
⟨gcpscad⟩ if (DT_tool_no >  0) { oopendxfDTfile(str(fn,".",DT_tool_no,".dxf"));
⟨gcpscad⟩     dxfpreamble(DT_tool_no);
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩
```

`writedxf` Once files have been opened they may be written to. There is a base command:

```
⟨gcpy⟩def writedxf(*arguments):
⟨gcpy⟩    line_to_write = ""
⟨gcpy⟩    for element in arguments:
⟨gcpy⟩        line_to_write += element
⟨gcpy⟩    dxf.write(line_to_write)
⟨gcpy⟩    dxf.write("\n")
⟨gcpy⟩
```

`writedxflgbl` `writedxflgsq` `writedxflgV` `writedxfsmbl` `writedxfsmsq` `writedxfsmV` `writedxfKH` `writedxfDT`

and for each tool/size combination, an appropriate command:

- Ball nose, large (lgbl)
- Ball nose, small (smbl)
- Square, large (lgsq)
- Square, small (smsq)
- V, large (lgV)
- V, small (smV)
- Keyhole (KH)

- Dovetail (DT)

```
⟨gcpy⟩def writedxflgbl(*arguments):
⟨gcpy⟩    line_to_write = ""
⟨gcpy⟩    for element in arguments:
⟨gcpy⟩        line_to_write += element
⟨gcpy⟩    dxflgbl.write(line_to_write)
⟨gcpy⟩    print(line_to_write)
⟨gcpy⟩    dxflgbl.write("\n")
⟨gcpy⟩
⟨gcpy⟩def writedxflgsq(*arguments):
⟨gcpy⟩    line_to_write = ""
⟨gcpy⟩    for element in arguments:
⟨gcpy⟩        line_to_write += element
⟨gcpy⟩    dxflgsq.write(line_to_write)
⟨gcpy⟩    print(line_to_write)
⟨gcpy⟩    dxflgsq.write("\n")
⟨gcpy⟩
⟨gcpy⟩def writedxflgV(*arguments):
⟨gcpy⟩    line_to_write = ""
⟨gcpy⟩    for element in arguments:
⟨gcpy⟩        line_to_write += element
⟨gcpy⟩    dxflgV.write(line_to_write)
⟨gcpy⟩    print(line_to_write)
⟨gcpy⟩    dxflgV.write("\n")
⟨gcpy⟩
⟨gcpy⟩def writedxfsmbl(*arguments):
⟨gcpy⟩    line_to_write = ""
⟨gcpy⟩    for element in arguments:
⟨gcpy⟩        line_to_write += element
⟨gcpy⟩    dxfsmbl.write(line_to_write)
⟨gcpy⟩    print(line_to_write)
⟨gcpy⟩    dxfsmbl.write("\n")
⟨gcpy⟩
⟨gcpy⟩def writedxfsmsq(*arguments):
⟨gcpy⟩    line_to_write = ""
⟨gcpy⟩    for element in arguments:
⟨gcpy⟩        line_to_write += element
⟨gcpy⟩    dxfsmsq.write(line_to_write)
⟨gcpy⟩    print(line_to_write)
⟨gcpy⟩    dxfsmsq.write("\n")
⟨gcpy⟩
⟨gcpy⟩def writedxfsmV(*arguments):
⟨gcpy⟩    line_to_write = ""
⟨gcpy⟩    for element in arguments:
⟨gcpy⟩        line_to_write += element
⟨gcpy⟩    dxfsmV.write(line_to_write)
⟨gcpy⟩    print(line_to_write)
⟨gcpy⟩    dxfsmV.write("\n")
⟨gcpy⟩
⟨gcpy⟩def writedxfKH(*arguments):
⟨gcpy⟩    line_to_write = ""
⟨gcpy⟩    for element in arguments:
⟨gcpy⟩        line_to_write += element
⟨gcpy⟩    dxfKH.write(line_to_write)
```

```
⟨gcpy⟩    print(line_to_write)
⟨gcpy⟩    dxfKH.write("\n")
⟨gcpy⟩
⟨gcpy⟩def writedxfDT(*arguments):
⟨gcpy⟩    line_to_write = ""
⟨gcpy⟩    for element in arguments:
⟨gcpy⟩        line_to_write += element
⟨gcpy⟩    dxfDT.write(line_to_write)
⟨gcpy⟩    print(line_to_write)
⟨gcpy⟩    dxfDT.write("\n")
⟨gcpy⟩
⟨gcpy⟩def writesvg(*arguments):
⟨gcpy⟩    line_to_write = ""
⟨gcpy⟩    for element in arguments:
⟨gcpy⟩        line_to_write += element
⟨gcpy⟩    svg.write(line_to_write)
⟨gcpy⟩    print(line_to_write)
⟨gcpy⟩
⟨gcpy⟩def pwritesvgline():
⟨gcpy⟩    svg.write("\n")
⟨gcpy⟩
```

owritecomment
dxfwriteone
dxfwritelgbl
dxfwritelgsq
dxfwritelgV
dxfwritesmbl
dxfwritesmsq
dxfwritesmV

Separate OpenSCAD modules will be used for either writing out comments in G-code (.nc) files or adding to a DXF file — for each different tool in a file there will be a matching module to write to it.

```
⟨pyscad⟩ module owritecomment(comment) {
⟨pyscad⟩  writeln("(",comment,")");
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module dxfwriteone(first) {
⟨pyscad⟩  writedxf(first);
⟨pyscad⟩ // writeln(first);
⟨pyscad⟩ // echo(first);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module dxfwritelgbl(first) {
⟨pyscad⟩  writedxflgbl(first);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module dxfwritelgsq(first) {
⟨pyscad⟩  writedxflgsq(first);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module dxfwritelgV(first) {
⟨pyscad⟩  writedxflgV(first);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module dxfwritesmbl(first) {
⟨pyscad⟩  writedxfsmbl(first);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module dxfwritesmsq(first) {
⟨pyscad⟩  writedxfsmsq(first);
⟨pyscad⟩ }
```

```
⟨pyscad⟩
⟨pyscad⟩ module dxfwritesmV(first) {
⟨pyscad⟩  writedxfsmV(first);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module dxfwriteKH(first) {
⟨pyscad⟩  writedxfKH(first);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module dxfwriteDT(first) {
⟨pyscad⟩  writedxfDT(first);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module svgwriteone(first) {
⟨pyscad⟩  writesvg(first);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module writesvglineend(first) {
⟨pyscad⟩  pwritesvgline();
⟨pyscad⟩ }
⟨pyscad⟩
```

Since it is not convenient to stitch together and then write out multiple elements, the most expedient thing to do is to have discrete commands for each possible number of arguments, one through thirteen.

```
⟨pyscad⟩ module owriteone(first) {
⟨pyscad⟩  writeln(first);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module owritetwo(first, second) {
⟨pyscad⟩  writeln(first, second);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module owritethree(first, second, third) {
⟨pyscad⟩  writeln(first, second, third);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module owritefour(first, second, third, fourth) {
⟨pyscad⟩  writeln(first, second, third, fourth);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module owritefive(first, second, third, fourth, fifth) {
⟨pyscad⟩  writeln(first, second, third, fourth, fifth);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module owritesix(first, second, third, fourth, fifth, sixth) {
⟨pyscad⟩  writeln(first, second, third, fourth, fifth, sixth);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module owriteseven(first, second, third, fourth, fifth, sixth, seventh) {
⟨pyscad⟩  writeln(first, second, third, fourth, fifth, sixth, seventh);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module owriteeight(first, second, third, fourth, fifth, sixth, seventh,eighth)
```

```
⟨pyscad⟩  writeln(first, second, third, fourth, fifth, sixth, seventh,eighth);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module owritenine(first, second, third, fourth, fifth, sixth, seventh, eighth, ninth) {
⟨pyscad⟩  writeln(first, second, third, fourth, fifth, sixth, seventh, eighth, ninth);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module owriteten(first, second, third, fourth, fifth, sixth, seventh, eighth, ninth, tent
⟨pyscad⟩  writeln(first, second, third, fourth, fifth, sixth, seventh, eighth, ninth, tenth);
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module owriteeleven(first, second, third, fourth, fifth, sixth, seventh, eighth, ninth, t
⟨pyscad⟩  writeln(first, second, third, fourth, fifth, sixth, seventh, eighth, ninth, tenth, eleve
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module owritetwelve(first, second, third, fourth, fifth, sixth, seventh, eighth, ninth, t
⟨pyscad⟩  writeln(first, second, third, fourth, fifth, sixth, seventh, eighth, ninth, tenth, eleve
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module owritethirteen(first, second, third, fourth, fifth, sixth, seventh, eighth, ninth,
⟨pyscad⟩  writeln(first, second, third, fourth, fifth, sixth, seventh, eighth, ninth, tenth, eleve
⟨pyscad⟩ }
⟨pyscad⟩
```

dxfwrite
dxfpreamble
writesvgline

The dxfwrite module requires that the tool number be passed in, and that value will be used to write out to the appropriate file.

```
⟨gcpscad⟩ module dxfwrite(tn,arg) {
⟨gcpscad⟩ if (tn == large_ball_tool_no) {
⟨gcpscad⟩     dxfwritelgbl(arg);}
⟨gcpscad⟩ if (tn == large_square_tool_no) {
⟨gcpscad⟩     dxfwritelgsq(arg);}
⟨gcpscad⟩ if (tn == large_V_tool_no) {
⟨gcpscad⟩     dxfwritelgV(arg);}
⟨gcpscad⟩ if (tn == small_ball_tool_no) {
⟨gcpscad⟩     dxfwritesmbl(arg);}
⟨gcpscad⟩ if (tn == small_square_tool_no) {
⟨gcpscad⟩     dxfwritesmsq(arg);}
⟨gcpscad⟩ if (tn == small_V_tool_no) {
⟨gcpscad⟩     dxfwritesmV(arg);}
⟨gcpscad⟩ if (tn == DT_tool_no) {
⟨gcpscad⟩     dxfwriteDT(arg);}
⟨gcpscad⟩ if (tn == KH_tool_no) {
⟨gcpscad⟩     dxfwriteKH(arg);}
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module dxfpreamble(tn) {
⟨gcpscad⟩ //    echo(str("dxfpreamble",small_square_tool_no));
⟨gcpscad⟩     dxfwrite(tn,"0");
⟨gcpscad⟩     dxfwrite(tn,"SECTION");
⟨gcpscad⟩     dxfwrite(tn,"2");
⟨gcpscad⟩     dxfwrite(tn,"ENTITIES");
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module writesvgline(bx,by,ex,ey) {
```

```
⟨gcpscad⟩ if (generatesvg == true) {
⟨gcpscad⟩     svgwriteone(str("<path d=",chr(34),"M",bx*3.77953," ",by*3.77953," L",ex*3
    width=",chr(34),"1",chr(34)," fill=",chr(34),"none",chr(34)," /> "));
⟨gcpscad⟩     }
⟨gcpscad⟩ }
⟨gcpscad⟩
```

beginpolyline
dxfbpl

Similarly, each each element which may be written to a DXF file will have a user module as well as an internal module which will be called by it so as to write to the file for the current tool.

There are two notable elements which may be written to a DXF:

- a line: LWPOLYLINE is one possible implementation
- ARC — a notable option would be for the arc to close on itself, creating a circle

Note that arcs of greater than 90 degrees are not rendered accurately, so, for the sake of precision, they should be limited to a swing of 90 degrees or less. Further note that 4 arcs may be stitched together to make a circle:

```
dxfarc(small\_square\_tool\_no,10,10,5,0,90);
dxfarc(small\_square\_tool\_no,10,10,5,90,180);
dxfarc(small\_square\_tool\_no,10,10,5,180,270);
dxfarc(small\_square\_tool\_no,10,10,5,270,360);
```

A further refinement would be to connect multiple line segments/arcs into a larger polyline, but since most CAM tools implicitly join elements on import, that is not necessary.

There are three possible interactions for DXF elements and toolpaths:

- describe the motion of the tool
- define a perimeter of an area which will be cut by a tool
- define a centerpoint for a specialty toolpath such as Drill or Keyhhole

and it is possible that multiple such elements could be instantiated for a given toolpath.

```
⟨gcpscad⟩ module dxfpl(tn,xbegin,ybegin,xend,yend) {
⟨gcpscad⟩     dxfwrite(tn,"0");
⟨gcpscad⟩     dxfwrite(tn,"LWPOLYLINE");
⟨gcpscad⟩     dxfwrite(tn,"90");
⟨gcpscad⟩     dxfwrite(tn,"2");
⟨gcpscad⟩     dxfwrite(tn,"70");
⟨gcpscad⟩     dxfwrite(tn,"0");
⟨gcpscad⟩     dxfwrite(tn,"43");
⟨gcpscad⟩     dxfwrite(tn,"0");
⟨gcpscad⟩     dxfwrite(tn,"10");
⟨gcpscad⟩     dxfwrite(tn,str(xbegin));
⟨gcpscad⟩     dxfwrite(tn,"20");
⟨gcpscad⟩     dxfwrite(tn,str(ybegin));
⟨gcpscad⟩     dxfwrite(tn,"10");
```

```
⟨gcpscad⟩     dxfwrite(tn,str(xend));
⟨gcpscad⟩     dxfwrite(tn,"20");
⟨gcpscad⟩     dxfwrite(tn,str(yend));
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module dxfpolyline(tn,xbegin,ybegin,xend,yend) {
⟨gcpscad⟩ if (generatedxf == true) {
⟨gcpscad⟩     dxfwriteone("0");
⟨gcpscad⟩     dxfwriteone("LWPOLYLINE");
⟨gcpscad⟩     dxfwriteone("90");
⟨gcpscad⟩     dxfwriteone("2");
⟨gcpscad⟩     dxfwriteone("70");
⟨gcpscad⟩     dxfwriteone("0");
⟨gcpscad⟩     dxfwriteone("43");
⟨gcpscad⟩     dxfwriteone("0");
⟨gcpscad⟩     dxfwriteone("10");
⟨gcpscad⟩     dxfwriteone(str(xbegin));
⟨gcpscad⟩     dxfwriteone("20");
⟨gcpscad⟩     dxfwriteone(str(ybegin));
⟨gcpscad⟩     dxfwriteone("10");
⟨gcpscad⟩     dxfwriteone(str(xend));
⟨gcpscad⟩     dxfwriteone("20");
⟨gcpscad⟩     dxfwriteone(str(yend));
⟨gcpscad⟩     dxfpl(tn,xbegin,ybegin,xend,yend);
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module dxfa(tn,xcenter,ycenter,radius,anglebegin,endangle) {
⟨gcpscad⟩     dxfwrite(tn,"0");
⟨gcpscad⟩     dxfwrite(tn,"ARC");
⟨gcpscad⟩     dxfwrite(tn,"10");
⟨gcpscad⟩     dxfwrite(tn,str(xcenter));
⟨gcpscad⟩     dxfwrite(tn,"20");
⟨gcpscad⟩     dxfwrite(tn,str(ycenter));
⟨gcpscad⟩     dxfwrite(tn,"40");
⟨gcpscad⟩     dxfwrite(tn,str(radius));
⟨gcpscad⟩     dxfwrite(tn,"50");
⟨gcpscad⟩     dxfwrite(tn,str(anglebegin));
⟨gcpscad⟩     dxfwrite(tn,"51");
⟨gcpscad⟩     dxfwrite(tn,str(endangle));
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module dxfarc(tn,xcenter,ycenter,radius,anglebegin,endangle) {
⟨gcpscad⟩ if (generatedxf == true) {
⟨gcpscad⟩     dxfwriteone("0");
⟨gcpscad⟩     dxfwriteone("ARC");
⟨gcpscad⟩     dxfwriteone("10");
⟨gcpscad⟩     dxfwriteone(str(xcenter));
⟨gcpscad⟩     dxfwriteone("20");
⟨gcpscad⟩     dxfwriteone(str(ycenter));
⟨gcpscad⟩     dxfwriteone("40");
⟨gcpscad⟩     dxfwriteone(str(radius));
⟨gcpscad⟩     dxfwriteone("50");
⟨gcpscad⟩     dxfwriteone(str(anglebegin));
```

```
⟨gcpscad⟩     dxfwriteone("51");
⟨gcpscad⟩     dxfwriteone(str(endangle));
⟨gcpscad⟩  dxfa(tn,xcenter,ycenter,radius,anglebegin,endangle);
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩
```

The original implementation of polylines worked, but may be removed.

```
⟨gcpscad⟩ module dxfbpl(tn,bx,by) {
⟨gcpscad⟩     dxfwrite(tn,"0");
⟨gcpscad⟩     dxfwrite(tn,"POLYLINE");
⟨gcpscad⟩     dxfwrite(tn,"8");
⟨gcpscad⟩     dxfwrite(tn,"default");
⟨gcpscad⟩     dxfwrite(tn,"66");
⟨gcpscad⟩     dxfwrite(tn,"1");
⟨gcpscad⟩     dxfwrite(tn,"70");
⟨gcpscad⟩     dxfwrite(tn,"0");
⟨gcpscad⟩     dxfwrite(tn,"0");
⟨gcpscad⟩     dxfwrite(tn,"VERTEX");
⟨gcpscad⟩     dxfwrite(tn,"8");
⟨gcpscad⟩     dxfwrite(tn,"default");
⟨gcpscad⟩     dxfwrite(tn,"70");
⟨gcpscad⟩     dxfwrite(tn,"32");
⟨gcpscad⟩     dxfwrite(tn,"10");
⟨gcpscad⟩     dxfwrite(tn,str(bx));
⟨gcpscad⟩     dxfwrite(tn,"20");
⟨gcpscad⟩     dxfwrite(tn,str(by));
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module beginpolyline(bx,by,bz) {
⟨gcpscad⟩ if (generatedxf == true) {
⟨gcpscad⟩     dxfwriteone("0");
⟨gcpscad⟩     dxfwriteone("POLYLINE");
⟨gcpscad⟩     dxfwriteone("8");
⟨gcpscad⟩     dxfwriteone("default");
⟨gcpscad⟩     dxfwriteone("66");
⟨gcpscad⟩     dxfwriteone("1");
⟨gcpscad⟩     dxfwriteone("70");
⟨gcpscad⟩     dxfwriteone("0");
⟨gcpscad⟩     dxfwriteone("0");
⟨gcpscad⟩     dxfwriteone("VERTEX");
⟨gcpscad⟩     dxfwriteone("8");
⟨gcpscad⟩     dxfwriteone("default");
⟨gcpscad⟩     dxfwriteone("70");
⟨gcpscad⟩     dxfwriteone("32");
⟨gcpscad⟩     dxfwriteone("10");
⟨gcpscad⟩     dxfwriteone(str(bx));
⟨gcpscad⟩     dxfwriteone("20");
⟨gcpscad⟩     dxfwriteone(str(by));
⟨gcpscad⟩     dxfbpl(current_tool(),bx,by);}
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module dxfapl(tn,bx,by) {
⟨gcpscad⟩     dxfwriteone("0");
```

```
⟨gcpscad⟩     dxfwrite(tn,"VERTEX");
⟨gcpscad⟩     dxfwrite(tn,"8");
⟨gcpscad⟩     dxfwrite(tn,"default");
⟨gcpscad⟩     dxfwrite(tn,"70");
⟨gcpscad⟩     dxfwrite(tn,"32");
⟨gcpscad⟩     dxfwrite(tn,"10");
⟨gcpscad⟩     dxfwrite(tn,str(bx));
⟨gcpscad⟩     dxfwrite(tn,"20");
⟨gcpscad⟩     dxfwrite(tn,str(by));
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module addpolyline(bx,by,bz) {
⟨gcpscad⟩ if (generatedxf == true) {
⟨gcpscad⟩     dxfwrite(tn,"0");
⟨gcpscad⟩     dxfwriteone("VERTEX");
⟨gcpscad⟩     dxfwriteone("8");
⟨gcpscad⟩     dxfwriteone("default");
⟨gcpscad⟩     dxfwriteone("70");
⟨gcpscad⟩     dxfwriteone("32");
⟨gcpscad⟩     dxfwriteone("10");
⟨gcpscad⟩     dxfwriteone(str(bx));
⟨gcpscad⟩     dxfwriteone("20");
⟨gcpscad⟩     dxfwriteone(str(by));
⟨gcpscad⟩     dxfapl(current_tool(),bx,by);
⟨gcpscad⟩     }
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module dxfcpl(tn) {
⟨gcpscad⟩     dxfwrite(tn,"0");
⟨gcpscad⟩     dxfwrite(tn,"SEQEND");
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module closepolyline() {
⟨gcpscad⟩ if (generatedxf == true) {
⟨gcpscad⟩     dxfwriteone("0");
⟨gcpscad⟩     dxfwriteone("SEQEND");
⟨gcpscad⟩     dxfcpl(current_tool());
⟨gcpscad⟩     }
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module writecomment(comment) {
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩  owritecomment(comment);
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩
```

`pclosegcodefile` `pclosesvgfile` `pclosedxffile`

At the end of the project it will be necessary to close each file. In some instances it will be necessary to write additional information, depending on the file format.

```
⟨gcpy⟩def pclosegcodefile():
⟨gcpy⟩    f.close()
⟨gcpy⟩
⟨gcpy⟩def pclosesvgfile():
```

```
⟨gcpy⟩    svg.close()
⟨gcpy⟩
⟨gcpy⟩def pclosedxffile():
⟨gcpy⟩    dxf.close()
⟨gcpy⟩
⟨gcpy⟩def pclosedxflgblfile():
⟨gcpy⟩    dxflgbl.close()
⟨gcpy⟩
⟨gcpy⟩def pclosedxflgsqfile():
⟨gcpy⟩    dxflgsq.close()
⟨gcpy⟩
⟨gcpy⟩def pclosedxflgVfile():
⟨gcpy⟩    dxflgV.close()
⟨gcpy⟩
⟨gcpy⟩def pclosedxfsmblfile():
⟨gcpy⟩    dxfsmbl.close()
⟨gcpy⟩
⟨gcpy⟩def pclosedxfsmsqfile():
⟨gcpy⟩    dxfsmsq.close()
⟨gcpy⟩
⟨gcpy⟩def pclosedxfsmVfile():
⟨gcpy⟩    dxfsmV.close()
⟨gcpy⟩
⟨gcpy⟩def pclosedxfDTfile():
⟨gcpy⟩    dxfDT.close()
⟨gcpy⟩
⟨gcpy⟩def pclosedxfKHfile():
⟨gcpy⟩    dxfKH.close()
⟨gcpy⟩
```

oclosegcodefile
oclosedxffile
oclosedxflgblfile

```
⟨pyscad⟩ module oclosegcodefile() {
⟨pyscad⟩  pclosegcodefile();
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oclosedxffile() {
⟨pyscad⟩  pclosedxffile();
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oclosedxflgblfile() {
⟨pyscad⟩  pclosedxflgblfile();
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oclosedxflgsqfile() {
⟨pyscad⟩  pclosedxflgsqfile();
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oclosedxflgVfile() {
⟨pyscad⟩  pclosedxflgVfile();
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oclosedxfsmblfile() {
⟨pyscad⟩  pclosedxfsmblfile();
⟨pyscad⟩ }
```

```
⟨pyscad⟩
⟨pyscad⟩ module oclosedxfsmsqfile() {
⟨pyscad⟩  pclosedxfsmsqfile();
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oclosedxfsmVfile() {
⟨pyscad⟩  pclosedxfsmVfile();
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oclosedxfDTfile() {
⟨pyscad⟩  pclosedxfDTfile();
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oclosedxfKHfile() {
⟨pyscad⟩  pclosedxfKHfile();
⟨pyscad⟩ }
⟨pyscad⟩
⟨pyscad⟩ module oclosesvgfile() {
⟨pyscad⟩  pclosesvgfile();
⟨pyscad⟩ }
⟨pyscad⟩
```

closegcodefile
dxfpostamble
closedxffile

```
⟨gcpscad⟩ module closegcodefile() {
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩     owriteone("M05");
⟨gcpscad⟩     owriteone("M02");
⟨gcpscad⟩  oclosegcodefile();
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module dxfpostamble(arg) {
⟨gcpscad⟩     dxfwrite(arg,"0");
⟨gcpscad⟩     dxfwrite(arg,"ENDSEC");
⟨gcpscad⟩     dxfwrite(arg,"0");
⟨gcpscad⟩     dxfwrite(arg,"EOF");
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module closedxffile() {
⟨gcpscad⟩ if (generatedxf == true) {
⟨gcpscad⟩     dxfwriteone("0");
⟨gcpscad⟩     dxfwriteone("ENDSEC");
⟨gcpscad⟩     dxfwriteone("0");
⟨gcpscad⟩     dxfwriteone("EOF");
⟨gcpscad⟩  oclosedxffile();
⟨gcpscad⟩     echo("CLOSING");
⟨gcpscad⟩ if (large_ball_tool_no >  0) { dxfpostamble(large_ball_tool_no);
⟨gcpscad⟩     oclosedxflgblfile();
⟨gcpscad⟩ }
⟨gcpscad⟩ if (large_square_tool_no >  0) { dxfpostamble(large_square_tool_no);
⟨gcpscad⟩     oclosedxflgsqfile();
⟨gcpscad⟩ }
⟨gcpscad⟩ if (large_V_tool_no >  0) { dxfpostamble(large_V_tool_no);
⟨gcpscad⟩     oclosedxflgVfile();
⟨gcpscad⟩ }
```

```
⟨gcpscad⟩ if (small_ball_tool_no >  0) { dxfpostamble(small_ball_tool_no);
⟨gcpscad⟩     oclosedxfsmblfile();
⟨gcpscad⟩ }
⟨gcpscad⟩ if (small_square_tool_no >  0) { dxfpostamble(small_square_tool_no);
⟨gcpscad⟩     oclosedxfsmsqfile();
⟨gcpscad⟩ }
⟨gcpscad⟩ if (small_V_tool_no >  0) { dxfpostamble(small_V_tool_no);
⟨gcpscad⟩     oclosedxfsmVfile();
⟨gcpscad⟩ }
⟨gcpscad⟩ if (DT_tool_no >  0) { dxfpostamble(DT_tool_no);
⟨gcpscad⟩     oclosedxfDTfile();
⟨gcpscad⟩ }
⟨gcpscad⟩ if (KH_tool_no >  0) { dxfpostamble(KH_tool_no);
⟨gcpscad⟩     oclosedxfKHfile();
⟨gcpscad⟩ }
⟨gcpscad⟩     }
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module closesvgfile() {
⟨gcpscad⟩ if (generatesvg == true) {
⟨gcpscad⟩     svgwriteone("</svg> ");
⟨gcpscad⟩  oclosesvgfile();
⟨gcpscad⟩     echo("CLOSING SVG");
⟨gcpscad⟩     }
⟨gcpscad⟩ }
⟨gcpscad⟩
```

2.4 Movement and Cutting

otm
ocut
orapid

With all the scaffolding in place, it is possible to model tool movement and cutting and to write out files which represent the desired machine motions.

```
⟨gcpscad⟩ module otm(ex, ey, ez, r,g,b) {
⟨gcpscad⟩ color([r,g,b]) hull(){
⟨gcpscad⟩     translate([xpos(), ypos(), zpos()]){
⟨gcpscad⟩       select_tool(current_tool());
⟨gcpscad⟩     }
⟨gcpscad⟩     translate([ex, ey, ez]){
⟨gcpscad⟩       select_tool(current_tool());
⟨gcpscad⟩     }
⟨gcpscad⟩   }
⟨gcpscad⟩ oset(ex, ey, ez);
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module ocut(ex, ey, ez) {
⟨gcpscad⟩ //color([0.2,1,0.2]) hull(){
⟨gcpscad⟩ otm(ex, ey, ez, 0.2,1,0.2);
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module orapid(ex, ey, ez) {
⟨gcpscad⟩ //color([0.93,0,0]) hull(){
⟨gcpscad⟩ otm(ex, ey, ez, 0.93,0,0);
⟨gcpscad⟩ }
⟨gcpscad⟩
```

```
⟨gcpscad⟩ module rapidbx(bx, by, bz, ex, ey, ez) {
⟨gcpscad⟩ // writeln("G0 X",bx," Y", by, "Z", bz);
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩  writecomment("rapid");
⟨gcpscad⟩  owritesix("G0 X",str(ex)," Y", str(ey), " Z", str(ez));
⟨gcpscad⟩ }
⟨gcpscad⟩     orapid(ex, ey, ez);
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module rapid(ex, ey, ez) {
⟨gcpscad⟩ // writeln("G0 X",bx," Y", by, "Z", bz);
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩  writecomment("rapid");
⟨gcpscad⟩  owritesix("G0 X",str(ex)," Y", str(ey), " Z", str(ez));
⟨gcpscad⟩ }
⟨gcpscad⟩     orapid(ex, ey, ez);
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module movetosafez() {
⟨gcpscad⟩ //this should be move to retract height
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩  writecomment("Move to safe Z to avoid workholding");
⟨gcpscad⟩     owriteone("G53G0Z-5.000");
⟨gcpscad⟩ }
⟨gcpscad⟩     orapid(getxpos(), getypos(), retractheight+55);
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module begintoolpath(bx,by,bz) {
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩  writecomment("PREPOSITION FOR RAPID PLUNGE");
⟨gcpscad⟩     owritefour("G0X", str(bx), "Y",str(by));
⟨gcpscad⟩     owritetwo("Z", str(bz));
⟨gcpscad⟩     }
⟨gcpscad⟩     orapid(bx,by,bz);
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module movetosafeheight() {
⟨gcpscad⟩ //this should be move to machine position
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩ // writecomment("PREPOSITION FOR RAPID PLUNGE");Z25.650
⟨gcpscad⟩ //G1Z24.663F381.0 ,"F",str(plunge)
⟨gcpscad⟩ if (zeroheight == "Top") {
⟨gcpscad⟩     owritetwo("Z",str(retractheight));
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩     orapid(getxpos(), getypos(), retractheight+55);
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module cutoneaxis_setfeed(axis,depth,feed) {
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩ // writecomment("PREPOSITION FOR RAPID PLUNGE");Z25.650
⟨gcpscad⟩ //G1Z24.663F381.0 ,"F",str(plunge) G1Z7.612F381.0
⟨gcpscad⟩ if (zeroheight == "Top") {
⟨gcpscad⟩     owritefive("G1",axis,str(depth),"F",str(feed));
```

```
⟨gcpscad⟩ }
⟨gcpscad⟩ }
⟨gcpscad⟩ if (axis == "X") {setxpos(depth);}
⟨gcpscad⟩ if (axis == "Y") {setypos(depth);}
⟨gcpscad⟩ if (axis == "Z") {setzpos(depth);}
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module cut(ex, ey, ez) {
⟨gcpscad⟩ // writeln("G0 X",bx," Y", by, "Z", bz);
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩ // writecomment("rapid");
⟨gcpscad⟩  owritesix("G1 X",str(ex)," Y", str(ey), " Z", str(ez));
⟨gcpscad⟩ }
⟨gcpscad⟩ //if (generatesvg == true) {
⟨gcpscad⟩ // owritesix("G1 X",str(ex)," Y", str(ey), " Z", str(ez));
⟨gcpscad⟩ //    orapid(getxpos(), getypos(), retractheight+5);
⟨gcpscad⟩ //    writesvgline(getxpos(),getypos(),ex,ey);
⟨gcpscad⟩ //}
⟨gcpscad⟩ ocut(ex, ey, ez);
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module cutwithfeed(ex, ey, ez, feed) {
⟨gcpscad⟩ // writeln("G0 X",bx," Y", by, "Z", bz);
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩ // writecomment("rapid");
⟨gcpscad⟩  owriteeight("G1 X",str(ex)," Y", str(ey), " Z", str(ez),"F",str(feed));
⟨gcpscad⟩ }
⟨gcpscad⟩ ocut(ex, ey, ez);
⟨gcpscad⟩ }
⟨gcpscad⟩
⟨gcpscad⟩ module endtoolpath() {
⟨gcpscad⟩ if (generategcode == true) {
⟨gcpscad⟩ //Z31.750
⟨gcpscad⟩ // owriteone("G53G0Z-5.000");
⟨gcpscad⟩     owritetwo("Z",str(retractheight));
⟨gcpscad⟩ }
⟨gcpscad⟩     orapid(getxpos(),getypos(),retractheight);
⟨gcpscad⟩ }
```

3 gcodepreview_template.scad

The commands may then be put together using a template which will ensure that the various files are used/included as necessary, that files are opened before being written to, and that they are closed at the end.

```
⟨gcptmpl⟩ //!OpenSCAD
⟨gcptmpl⟩
⟨gcptmpl⟩ use <gcodepreview.py>;
⟨gcptmpl⟩ use <pygcodepreview.scad>;
⟨gcptmpl⟩ include <gcodepreview.scad>;
⟨gcptmpl⟩
⟨gcptmpl⟩ $fa = 2;
⟨gcptmpl⟩ $fs = 0.125;
⟨gcptmpl⟩
```

```
⟨gcptmpl⟩ /* [Export] */
⟨gcptmpl⟩ Base_filename = "export";
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Export] */
⟨gcptmpl⟩ generatedxf = true;
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Export] */
⟨gcptmpl⟩ generategcode = true;
⟨gcptmpl⟩
⟨gcptmpl⟩ ///* [Export] */
⟨gcptmpl⟩ //generatesvg = false;
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [CAM] */
⟨gcptmpl⟩ toolradius = 1.5875;
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [CAM] */
⟨gcptmpl⟩ large_ball_tool_no = 0; // [0:0,111:111,101:101,202:202]
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [CAM] */
⟨gcptmpl⟩ large_square_tool_no = 0; // [0:0,112:112,102:102,201:201]
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [CAM] */
⟨gcptmpl⟩ large_V_tool_no = 0; // [0:0,301:301,690:690]
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [CAM] */
⟨gcptmpl⟩ small_ball_tool_no = 0; // [0:0,121:121,111:111,101:101]
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [CAM] */
⟨gcptmpl⟩ small_square_tool_no = 102; // [0:0,122:122,112:112,102:102]
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [CAM] */
⟨gcptmpl⟩ small_V_tool_no = 0; // [0:0,390:390,301:301]
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [CAM] */
⟨gcptmpl⟩ KH_tool_no = 0; // [0:0,375:375]
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [CAM] */
⟨gcptmpl⟩ DT_tool_no = 0; // [0:0,814:814]
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Feeds and Speeds] */
⟨gcptmpl⟩ plunge = 100;
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Feeds and Speeds] */
⟨gcptmpl⟩ feed = 400;
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Feeds and Speeds] */
⟨gcptmpl⟩ speed = 16000;
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Feeds and Speeds] */
⟨gcptmpl⟩ square_ratio = 1.0; // [0.25:2]
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Feeds and Speeds] */
⟨gcptmpl⟩ small_V_ratio = 0.75; // [0.25:2]
⟨gcptmpl⟩
```

```
⟨gcptmpl⟩ /* [Feeds and Speeds] */
⟨gcptmpl⟩ large_V_ratio = 0.875; // [0.25:2]
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Stock] */
⟨gcptmpl⟩ stocklength = 219;
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Stock] */
⟨gcptmpl⟩ stockwidth = 150;
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Stock] */
⟨gcptmpl⟩ stockthickness = 8.35;
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Stock] */
⟨gcptmpl⟩ zeroheight = "Top"; // [Top, Bottom]
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Stock] */
⟨gcptmpl⟩ stockorigin = "Center"; // [Lower-Left, Center-Left, Top-Left, Center]
⟨gcptmpl⟩
⟨gcptmpl⟩ /* [Stock] */
⟨gcptmpl⟩ retractheight = 9;
⟨gcptmpl⟩
⟨gcptmpl⟩ filename_gcode = str(Base_filename, ".nc");
⟨gcptmpl⟩ filename_dxf = str(Base_filename);
⟨gcptmpl⟩ //filename_svg = str(Base_filename, ".svg");
⟨gcptmpl⟩
⟨gcptmpl⟩ opengcodefile(filename_gcode);
⟨gcptmpl⟩ opendxffile(filename_dxf);
⟨gcptmpl⟩
⟨gcptmpl⟩ difference() {
⟨gcptmpl⟩ setupstock(stocklength, stockwidth, stockthickness, zeroheight, stockorigin);
⟨gcptmpl⟩
⟨gcptmpl⟩ movetosafez();
⟨gcptmpl⟩
⟨gcptmpl⟩ toolchange(small_square_tool_no,speed * square_ratio);
⟨gcptmpl⟩
⟨gcptmpl⟩ begintoolpath(0,0,0.25);
⟨gcptmpl⟩ beginpolyline(0,0,0.25);
⟨gcptmpl⟩
⟨gcptmpl⟩ cutoneaxis_setfeed("Z",0,plunge*square_ratio);
⟨gcptmpl⟩
⟨gcptmpl⟩ cutwithfeed(stocklength/2,stockwidth/2,-stockthickness,feed);
⟨gcptmpl⟩ addpolyline(stocklength/2,stockwidth/2,-stockthickness);
⟨gcptmpl⟩
⟨gcptmpl⟩ endtoolpath();
⟨gcptmpl⟩ closepolyline();
⟨gcptmpl⟩ }
⟨gcptmpl⟩
⟨gcptmpl⟩ closegcodefile();
⟨gcptmpl⟩ closedxffile();
```

4 cut2Dshapes and expansion

New features will be tried out in a file such as `cut2Dshapes.scad` insofar as the file structures will allow (tool definitions for example will need to consolidated in 2.2.1 which will need to be included in the projects which will make use of said features until such time as they are added into the main `gcodepreview.scad` file.

A basic requirement will be to define two-dimensional regions so as to cut them out. Two different geometric treatments will be necessary: modeling the geometry which defines the region to be cut out (output as a DXF); and modeling the movement of the tool, the toolpath which will be use in creating the 3D model and outputting the G-code.

In the TUG presentation/paper: http://tug.org/TUGboat/tb40-2/tb125adams-3d.pdf a list of 2D shapes was put forward — which of these will need to be created, or if some more general solution will be put forward is uncertain. For the time being, shapes will be implemented on an as-needed basis, as modified by the interaction with the requirements of toolpaths.

The program Carbide Create has toolpath types and options which are as follows:

- Contour — No Offset — the default, this is already supported in the existing code
- Contour — Outside Offset
- Contour — Inside Offset
- (Rectangular) Pocket — such toolpaths/geometry should include the rounding of the tool at the corners
- Drill — note that this is implemented as the plunging of a tool centered on a circle and normally that circle is the same diameter as the tool which is used.
- Keyhole — also beginning from a circle, a nice feature for this would be to include/model the areas which should be cleared for the sake of reducing wear on the tool and ensuring chip clearance

Some further considerations:

- relationship of geometry to toolpath — arguably there should be an option for each toolpath (we will use Carbide Create as a reference implementation) which is to be supported. Note that there are several possibilities: modeling the tool movement, describing the outline which the tool will cut, modeling a reference shape for the toolpath
- tool geometry — it should be possible to include support for specialty tooling such as dovetail cutters and to get an accurate 3D model, esp. for tooling which undercuts since they cannot be modeled in Carbide Create.
- feeds and speeds — if outputting G-code it would be nice to be able to import feeds and speeds from external files such as the .csv files used for user tool libraries in Carbide Create

- Starting and Max Depth — are there CAD programs which will make use of Z-axis information in a DXF? — would it be possible/necessary to further differentiate the DXF geometry? (currently written out separately for each toolpath in addition to one combined file)

4.1 Arcs for toolpaths and DXFs

A further consideration here is that G-code supports arcs in addition to the lines and polylines already implemented. Implementing arcs wants at least the following options for quadrant and direction:

- cutarcNWCW — cut the upper-left quadrant of a circle moving clockwise
- cutarcNWCC — upper-left quadrant counter-clockwise
- cutarcNECW
- cutarcNECC
- cutarcSECW
- cutarcSECC
- cutarcNECW
- cutarcNECC
- cutcircleCW — while it won't matter for generating a DXF, when G-code is implemented direction of cut will be a consideration for that
- cutcircleCCdxf

It will be necessary to have two separate representations of arcs — the DXF may be easily and directly supported with a single command, but representing the matching tool movement in OpenSCAD will require a series of short line movements which approximate the arc. At this time, the current version of Carbide Create only imports circles in DXF as curves, any other example is converted into polylines — unfortunately, the implementation of this is not such as would allow directly matching that representation. A work-around to import a DXF as curves is to convert the arc into a reasonable number of line segments so as to approximate the arc.

Note that there are the following representations/interfaces for representing an arc:

- G-code — G2 (clockwise) and G3 (counter-clockwise) arcs may be specified, and since the endpoint is the positional requirement, it is most likely best to use the offset to the center (I and J), rather than the radius parameter (K) `G2/3 ...`
- DXF — `dxfarc(tn,xcenter,ycenter,radius,anglebegin,endangle)`
- approximation of arc using lines (OpenSCAD) — note that this may also be used in DXF so as to sidestep the question of how many line segments there would be for a given arc representation

Cutting the quadrant arcs will greatly simplify the calculation and interface for the modules. A full set of 8 will be necessary, then circles may either be stitched together manually or a pair of modules made for them.

At this time, despite what the module names imply (`cutarcNWCWdxf`, &c.), only cutting and DXF generation is supported. Adding support for G-code will be done at a later time. Since these modules will ultimately support G-code, the interface will assume the stored `xpos` and `ypos` as the origin. Parameters which will need to be passed in are:

- `tn`
- `ex`
- `ey`
- `ez` — allowing a different Z position will make possible threading and similar helical toolpaths
- `xcenter` — the center position will be specified as an absolute position which will require calculating the offset when it is used for G-code's `IJ`, for which `xctr/yctr` are suggested
- `ycenter`
- `radius` — while this could be calculated, passing it in as a parameter is both convenient and acts as a check on the other parameters

Adding a simple loop to handle the processing of the `cut()` toolpaths affords a single point of control for adding additional features such as allowing the depth to vary as one cuts along an arc (two when the need to have a version which steps down):

```
⟨cut2D⟩ //!OpenSCAD
⟨cut2D⟩
⟨cut2D⟩ module arcloop(barc,earc, xcenter, ycenter, radius) {
⟨cut2D⟩   for (i = [barc : abs(1) : earc]) {
⟨cut2D⟩         cut(xcenter + radius * cos(i),
⟨cut2D⟩         ycenter + radius * sin(i),
⟨cut2D⟩         getzpos()-(gettzpos())
⟨cut2D⟩         );
⟨cut2D⟩     setxpos(xcenter + radius * cos(i));
⟨cut2D⟩     setypos(ycenter + radius * sin(i));
⟨cut2D⟩   }
⟨cut2D⟩ }
⟨cut2D⟩
⟨cut2D⟩ module narcloop(barc,earc, xcenter, ycenter, radius) {
⟨cut2D⟩   for (i = [barc : -1 : earc]) {
⟨cut2D⟩         cut(xcenter + radius * cos(i),
⟨cut2D⟩         ycenter + radius * sin(i),
⟨cut2D⟩         getzpos()-(gettzpos())
⟨cut2D⟩         );
⟨cut2D⟩     setxpos(xcenter + radius * cos(i));
⟨cut2D⟩     setypos(ycenter + radius * sin(i));
⟨cut2D⟩   }
⟨cut2D⟩ }
⟨cut2D⟩
```

The various textual versions are quite obvious:

```
⟨cut2D⟩ module cutarcNECCdxf(tn, ex, ey, ez, xcenter, ycenter, radius) {
⟨cut2D⟩   dxfarc(tn,xcenter,ycenter,radius,0,90);
⟨cut2D⟩   settzpos((getzpos()-ez)/90);
⟨cut2D⟩     arcloop(1,90, xcenter, ycenter, radius);
⟨cut2D⟩ }
⟨cut2D⟩
⟨cut2D⟩ module cutarcNWCCdxf(tn, ex, ey, ez, xcenter, ycenter, radius) {
⟨cut2D⟩   dxfarc(tn,xcenter,ycenter,radius,90,180);
⟨cut2D⟩   settzpos((getzpos()-ez)/90);
⟨cut2D⟩     arcloop(91,180, xcenter, ycenter, radius);
⟨cut2D⟩ }
⟨cut2D⟩
⟨cut2D⟩ module cutarcSWCCdxf(tn, ex, ey, ez, xcenter, ycenter, radius) {
⟨cut2D⟩   dxfarc(tn,xcenter,ycenter,radius,180,270);
⟨cut2D⟩   settzpos((getzpos()-ez)/90);
⟨cut2D⟩     arcloop(181,270, xcenter, ycenter, radius);
⟨cut2D⟩ }
⟨cut2D⟩
⟨cut2D⟩ module cutarcSECCdxf(tn, ex, ey, ez, xcenter, ycenter, radius) {
⟨cut2D⟩   dxfarc(tn,xcenter,ycenter,radius,270,360);
⟨cut2D⟩   settzpos((getzpos()-ez)/90);
⟨cut2D⟩     arcloop(271,360, xcenter, ycenter, radius);
⟨cut2D⟩ }
⟨cut2D⟩
⟨cut2D⟩ module cutarcNECWdxf(tn, ex, ey, ez, xcenter, ycenter, radius) {
⟨cut2D⟩   dxfarc(tn,xcenter,ycenter,radius,0,90);
⟨cut2D⟩   settzpos((getzpos()-ez)/90);
⟨cut2D⟩     narcloop(89,0, xcenter, ycenter, radius);
⟨cut2D⟩ }
⟨cut2D⟩
⟨cut2D⟩ module cutarcSECWdxf(tn, ex, ey, ez, xcenter, ycenter, radius) {
⟨cut2D⟩   dxfarc(tn,xcenter,ycenter,radius,270,360);
⟨cut2D⟩   settzpos((getzpos()-ez)/90);
⟨cut2D⟩     narcloop(359,270, xcenter, ycenter, radius);
⟨cut2D⟩ }
⟨cut2D⟩
⟨cut2D⟩ module cutarcSWCWdxf(tn, ex, ey, ez, xcenter, ycenter, radius) {
⟨cut2D⟩   dxfarc(tn,xcenter,ycenter,radius,180,270);
⟨cut2D⟩   settzpos((getzpos()-ez)/90);
⟨cut2D⟩     narcloop(269,180, xcenter, ycenter, radius);
⟨cut2D⟩ }
⟨cut2D⟩
⟨cut2D⟩ module cutarcNWCWdxf(tn, ex, ey, ez, xcenter, ycenter, radius) {
⟨cut2D⟩   dxfarc(tn,xcenter,ycenter,radius,90,180);
⟨cut2D⟩   settzpos((getzpos()-ez)/90);
⟨cut2D⟩     narcloop(179,90, xcenter, ycenter, radius);
⟨cut2D⟩ }
⟨cut2D⟩
```

4.2 Keyhole toolpath and undercut tooling

The most topologically interesting toolpath is "Keyhole" — where other toolpaths have a direct correspondence between the associated geometry and the area cut, that Keyhole toolpaths may be used with tooling which undercuts will result in the creation of two different physical physical regions: the visible surface matching the union of the tool perimeter at the entry point and the linear movement of the shaft and the larger region of the tool perimeter at the depth which the tool is plunged to and moved along.

Tooling for such toolpaths is defined at paragraph 2.2.1.2

Due to the possibility of rotation, for the in-between positions there are more cases than one would think — for each quadrant there are the following possibilities:

- one node on the clockwise side is outside of the quadrant
- two nodes on the clockwise side are outside of the quadrant
- all nodes are w/in the quadrant
- one node on the counter-clockwise side is outside of the quadrant
- two nodes on the counter-clockwise side are outside of the quadrant

Supporting all of these would require trigonometric comparisons in the If else blocks, so only the 4 quadrants, N, W, S, and E will be supported in the initial version. This will be done by wrapping the command with a version which only accepts those options:

```
⟨cut2D⟩ module keyhole_toolpath(kh_tool_no, kh_start_depth, kh_max_depth, kht_angle, kh_length) {
⟨cut2D⟩ if (kht_angle == "N") {
⟨cut2D⟩   keyhole_toolpath_degrees(kh_tool_no, kh_start_depth, kh_max_depth, 90, kh_length);
⟨cut2D⟩     } else if (kht_angle == "S") {
⟨cut2D⟩   keyhole_toolpath_degrees(kh_tool_no, kh_start_depth, kh_max_depth, 270, kh_length);
⟨cut2D⟩     } else if (kht_angle == "E") {
⟨cut2D⟩   keyhole_toolpath_degrees(kh_tool_no, kh_start_depth, kh_max_depth, 0, kh_length);
⟨cut2D⟩     } else if (kht_angle == "W") {
⟨cut2D⟩   keyhole_toolpath_degrees(kh_tool_no, kh_start_depth, kh_max_depth, 180, kh_length);
⟨cut2D⟩     }
⟨cut2D⟩ }
```

The original version of the command is renamed and called by that. Note that code is still present for the partial calculation of one quadrant (for the case of all nodes within the quadrant).

The first task is to place a circle at the origin which is invariant of angle:

```
⟨cut2D⟩ module keyhole_toolpath_degrees(kh_tool_no, kh_start_depth, kh_max_depth, kh_angle, kh_len
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,0,9
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,90,
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,180
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,270
```

Then it will be necessary to test for each possible case in a series of If Else blocks:

```
⟨cut2D⟩   if (kh_angle == 0) {
```

```
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))
asin((tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)/(tool_diameter(KH_tool_no, (kh_
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos()+kh_length,getypos(),tool_diameter(KH_tool_no, (kh_m
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos()+kh_length,getypos(),tool_diameter(KH_tool_no, (kh_m
⟨cut2D⟩ dxfpolyline(KH_tool_no,
⟨cut2D⟩  getxpos()+sqrt((tool_diameter(KH_tool_no, (kh_max_depth))/2)^2-
(tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)^2),
⟨cut2D⟩  getypos()+tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,
⟨cut2D⟩  getxpos()+kh_length,
⟨cut2D⟩  getypos()+tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2);
⟨cut2D⟩ dxfpolyline(KH_tool_no,
⟨cut2D⟩  getxpos()+sqrt((tool_diameter(KH_tool_no, (kh_max_depth))/2)^2-
(tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)^2),
⟨cut2D⟩  getypos()-tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,
⟨cut2D⟩  getxpos()+kh_length,
⟨cut2D⟩  getypos()-tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2);
⟨cut2D⟩ dxfpolyline(KH_tool_no,getxpos(),getypos(),getxpos()+kh_length,getypos());
⟨cut2D⟩  cutwithfeed(getxpos()+kh_length,getypos(),-kh_max_depth,feed);
⟨cut2D⟩  setxpos(getxpos()-kh_length);
⟨cut2D⟩   } else if (kh_angle > 0 && kh_angle < 90) {
⟨cut2D⟩ echo(kh_angle);
⟨cut2D⟩   dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth
⟨cut2D⟩   dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))
asin((tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)/(tool_diameter(KH_tool_no, (kh_
⟨cut2D⟩ dxfarc(KH_tool_no,
⟨cut2D⟩   getxpos()+(kh_length*cos(kh_angle)),
⟨cut2D⟩   getypos()+(kh_length*sin(kh_angle)),tool_diameter(KH_tool_no, (kh_max_depth+4
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos()+(kh_length*cos(kh_angle)),getypos()+(kh_length*sin(
⟨cut2D⟩ dxfpolyline(KH_tool_no,
⟨cut2D⟩  getxpos()+tool_diameter(KH_tool_no, (kh_max_depth))/2*cos(kh_angle+asin((tool_
⟨cut2D⟩  getypos()+tool_diameter(KH_tool_no, (kh_max_depth))/2*sin(kh_angle+asin((tool_
⟨cut2D⟩  getxpos()+(kh_length*cos(kh_angle))-((tool_diameter(KH_tool_no, (kh_max_depth+
⟨cut2D⟩  getypos()+(kh_length*sin(kh_angle))+((tool_diameter(KH_tool_no, (kh_max_depth+
⟨cut2D⟩ echo("a",tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2);
⟨cut2D⟩ echo("c",tool_diameter(KH_tool_no, (kh_max_depth))/2);
⟨cut2D⟩ echo("Aangle",asin((tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)/(tool_dia
⟨cut2D⟩ echo(kh_angle);
⟨cut2D⟩  cutwithfeed(getxpos()+(kh_length*cos(kh_angle)),getypos()+(kh_length*sin(kh_an
kh_max_depth,feed);
⟨cut2D⟩  setxpos(getxpos()-(kh_length*cos(kh_angle)));
⟨cut2D⟩  setypos(getypos()-(kh_length*sin(kh_angle)));
⟨cut2D⟩   } else if (kh_angle == 90) {
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))
asin(
⟨cut2D⟩     (tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)/(tool_diameter(KH_tool_n
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))
⟨cut2D⟩     (tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)/(tool_diameter(KH_tool_n
```

```
⟨cut2D⟩  dxfpolyline(KH_tool_no,getxpos(),getypos(),getxpos(),getypos()+kh_length);
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos()+kh_length,tool_diameter(KH_tool_no, (kh_max_depth+4.
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos()+kh_length,tool_diameter(KH_tool_no, (kh_max_depth+4.
⟨cut2D⟩  dxfpolyline(KH_tool_no,getxpos()+tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,getypos
(tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)^2),getxpos()+tool_diameter(KH_tool_no, (kh_max
⟨cut2D⟩  dxfpolyline(KH_tool_no,getxpos()-tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,getypos
(tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)^2),getxpos()-tool_diameter(KH_tool_no, (kh_max
⟨cut2D⟩  cutwithfeed(getxpos(),getypos()+kh_length,-kh_max_depth,feed);
⟨cut2D⟩  setypos(getypos()-kh_length);
⟨cut2D⟩   } else if (kh_angle == 180) {
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))/2,0,90);
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))/2,270,360)
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))/2,90,180-
asin((tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)/(tool_diameter(KH_tool_no, (kh_max_depth)
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))/2,180+asin
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos()-kh_length,getypos(),tool_diameter(KH_tool_no, (kh_max_depth+4.
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos()-kh_length,getypos(),tool_diameter(KH_tool_no, (kh_max_depth+4.
⟨cut2D⟩ dxfpolyline(KH_tool_no,
⟨cut2D⟩  getxpos()-sqrt((tool_diameter(KH_tool_no, (kh_max_depth))/2)^2-
(tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)^2),
⟨cut2D⟩  getypos()+tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,
⟨cut2D⟩  getxpos()-kh_length,
⟨cut2D⟩  getypos()+tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2);
⟨cut2D⟩ dxfpolyline(KH_tool_no,
⟨cut2D⟩  getxpos()-sqrt((tool_diameter(KH_tool_no, (kh_max_depth))/2)^2-
(tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)^2),
⟨cut2D⟩  getypos()-tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,
⟨cut2D⟩  getxpos()-kh_length,
⟨cut2D⟩  getypos()-tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2);
⟨cut2D⟩  dxfpolyline(KH_tool_no,getxpos(),getypos(),getxpos()-kh_length,getypos());
⟨cut2D⟩  cutwithfeed(getxpos()-kh_length,getypos(),-kh_max_depth,feed);
⟨cut2D⟩  setxpos(getxpos()+kh_length);
⟨cut2D⟩   } else if (kh_angle == 270) {
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))/2,0,90);
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))/2,90,180);
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))/2,270+asin
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos(),tool_diameter(KH_tool_no, (kh_max_depth))/2,180, 270
asin((tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2)/(tool_diameter(KH_tool_no, (kh_max_depth)
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos()-kh_length,tool_diameter(KH_tool_no, (kh_max_depth+4.
⟨cut2D⟩ dxfarc(KH_tool_no,getxpos(),getypos()-kh_length,tool_diameter(KH_tool_no, (kh_max_depth+4.
⟨cut2D⟩  dxfpolyline(KH_tool_no,getxpos()+tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,getypos
sqrt((tool_diameter(KH_tool_no, (kh_max_depth))/2)^2-(tool_diameter(KH_tool_no, (kh_max_depth+4.3
kh_length);
⟨cut2D⟩  dxfpolyline(KH_tool_no,getxpos()-tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,getypos
sqrt((tool_diameter(KH_tool_no, (kh_max_depth))/2)^2-(tool_diameter(KH_tool_no, (kh_max_depth+4.3
tool_diameter(KH_tool_no, (kh_max_depth+4.36))/2,getypos()-kh_length);
⟨cut2D⟩  dxfpolyline(KH_tool_no,getxpos(),getypos(),getxpos(),getypos()-
kh_length);
⟨cut2D⟩  cutwithfeed(getxpos(),getypos()-kh_length,-kh_max_depth,feed);
⟨cut2D⟩  setypos(getypos()+kh_length);
⟨cut2D⟩   }
⟨cut2D⟩ }
```

4.3 Bézier curves in 3 dimensions

One question is how many Bézier curves would it be necessary to have to define a surface in 3 dimensions. Attributes for this which are desirable/necessary:

- concise — a given Bézier curve should be represented by just the point coordinates, so two on-curve points, two off-curve points, each with a pair of coordinates
- For a given shape/region it will need to be possible to have a matching definition exactly match up with it so that one could piece together a larger more complex shape from smaller/simpler regions
- similarly it will be necessary for it to be possible to sub-divide a defined region — for example it should be possible if one had 4 adjacent regions, then the four quadrants at the intersection of the four regions could be used to construct a new region — is it possible to derive a new Bézier curve from half of two other curves?

For the three planes:

- XY
- XZ
- ZY

it should be possible to have three Bézier curves (left-most/right-most or front-back or top/bottom for two, and a mid-line for the third), so a region which can be so represented would be definable by:

```
3 planes * 3 Béziers * (2 on-curve + 2 off-curve points) == 36 coordinate pairs
```

which is a marked contrast to representations such as:
https://github.com/DavidPhillipOster/Teapot
and regions which could not be so represented could be sub-divided until the representation is workable.

4.4 Shapes and tool movement

5 Other Resources

Holidays are from https://nationaltoday.com/

References

[ConstGeom] Walmsley, Brian. *Construction Geometry.* 2d ed., Centennial College Press, 1981.

[MkCalc] Horvath, Joan, and Rich Cameron. *Make: Calculus: Build models to learn, visualize, and explore.* First edition., Make: Community LLC, 2022.

[MkGeom] Horvath, Joan, and Rich Cameron. *Make: Geometry: Learn by 3D Printing, Coding and Exploring*. First edition., Make: Community LLC, 2021.

[MkTrig] Horvath, Joan, and Rich Cameron. *Make: Trigonometry: Build your way from triangles to analytic geometry*. First edition., Make: Community LLC, 2023.

[PractShopMath] Begnal, Tom. *Practical Shop Math: Simple Solutions to Workshop Fractions, Formulas + Geometric Shapes*. Updated edition, Spring House Press, 2018.

[RS274] Thomas R. Kramer, Frederick M. Proctor, Elena R. Messina.
https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=823374
https://www.nist.gov/publications/nist-rs274ngc-interpreter-version-3